Università degli Studi di Milano - Bicocca
**Scuola di Scienze**
**Dipartimento di Informatica, Sistemistica e Comunicazione**
**Corso di Laurea Triennale in Informatica**

# Pipeline di Continuous Integration per Sistemi Cyber-Fisici

**Relatore:** Prof. Giovanni Denaro

**Co-relatore:** Dott. Jacopo Maltagliati

**Relazione della prova finale di:**
Alessandro Condello
Matricola 887918

**Anno Accademico 2026-2027**

# Ringraziamenti

Desidero ringraziare il Prof. Giovanni Denaro e il Dott. Jacopo Maltagliati per avermi permesso di sviluppare questo progetto.

Un sentito ringraziamento va alla mia famiglia, per aver creduto nelle mie capacità e per avermi sostenuto lungo tutto il mio percorso. In particolare mia madre Antonella, che non ha mai smesso di riporre fiducia in me, soprattutto nei momenti di maggiore difficoltà: senza il suo fondamentale sostegno non sarei riuscito a superare nemmeno il primo anno.

Ai miei amici, per aver reso le lezioni più leggere e le giornate di studio più piacevoli.

Questo traguardo non è soltanto un mio merito, ma il risultato del contributo di tutte le persone che mi sono state accanto nella vita.

# Indice

**Introduzione** **2**

**Glossario** **4**

**1 Analisi del Problema** **6**
1.1 Problema e Contesto . . . . . 6
1.2 Requisiti del Sistema . . . . . 6
1.2.1 Attori e Modalità di Interazione . . . . . 6
1.2.2 Requisiti Funzionali . . . . . 7
1.2.3 Requisiti Non Funzionali . . . . . 7
1.3 Soluzione Proposta . . . . . 7
1.3.1 Artifact Repository . . . . . 8
1.3.2 TestSequence Repository . . . . . 9
1.4 Soluzioni alternative . . . . . 10
1.4.1 JFrog e Sonatype . . . . . 10
1.4.2 Flask . . . . . 10
1.4.3 Tabella relazionale unica . . . . . 11

**2 Architettura e Pipeline** **12**
2.1 Panoramica Generale . . . . . 12
2.2 Back-end . . . . . 13
2.2.1 Tecnologie Utilizzate . . . . . 13
2.2.2 Architettura del Database . . . . . 13
2.2.3 Architettura del Back-end . . . . . 16
2.3 Front-end . . . . . 17
2.3.1 Tecnologie utilizzate . . . . . 18
2.3.2 Architettura del Front-end . . . . . 18
2.4 Architettura a Tre Tier . . . . . 19

**3 Back-end** **22**
3.1 Librerie e Dipendenze . . . . . 22
3.2 Chiamate API . . . . . 22
3.2.1 Inizializzazione del Servizio . . . . . 23
3.2.2 Standardizzazione dei Valori di Ritorno . . . . . 24

3.2.3 Standard API . . . . . 24
3.3 Artifact e TestSequence Repository . . . . . 25
3.3.1 Valori di Ritorno . . . . . 25
3.4 Gestione della Persistenza Relazionale . . . . . 26
3.4.1 Inizializzazione dello Schema Database . . . . . 26
3.4.2 Gestione della Concorrenza e delle Race Condition . . . . . 28
3.4.3 Analisi Operativa per Casistica . . . . . 29
3.5 Gestione del File System . . . . . 32
3.5.1 Concorrenza di I/O sul File System . . . . . 32
3.5.2 Specifica Dettagliata delle Funzioni . . . . . 34

**4 Front-end** **37**
4.1 Librerie e Dipendenze . . . . . 37
4.2 Model . . . . . 38
4.2.1 Modulo API Client . . . . . 38
4.2.2 Gestione dello Stato con `store.js` . . . . . 38
4.3 View . . . . . 43
4.3.1 Dashboard . . . . . 44
4.4 Controller . . . . . 47
4.4.1 Main Controller . . . . . 47
4.4.2 Handlers . . . . . 48

**5 Conclusioni** **51**
5.1 Sviluppi Futuri . . . . . 51

**A API Artifact Repository (/ar/)** **53**
A.1 Dashboard . . . . . 53
A.2 Aggiunta Artifact / File . . . . . 53
A.3 Rinomina Repository . . . . . 54
A.4 Ricerca e Recupero File Artifact . . . . . 54
A.5 Lista Tutti gli Artifact . . . . . 54
A.6 Data di Creazione Artifact . . . . . 55
A.7 Creatore Artifact . . . . . 55
A.8 Elenco File in Artifact . . . . . 55
A.9 Download Repository . . . . . 55
A.10 Download Singolo File . . . . . 56
A.11 Download Artifact per Nome . . . . . 56
A.12 Rimozione File / Artifact . . . . . 56

**B Test Sequence Repository (/ts/)** **57**
B.1 Dashboard Test . . . . . 57
B.2 Aggiunta Test Sequence . . . . . 57
B.3 Eliminazione Test Sequence . . . . . 58
B.4 Eliminazione Associazione Test Sequence e Feature Test . . . . . 58
B.5 Aggiunta Feature Test Case . . . . . 58

B.6 Associazione Requisito e Feature Case . . . . . 58
B.7 Eliminazione Associazione Requisito e Feature Case . . . . . 59
B.8 Associazione Test Sequence e Feature Case . . . . . 59
B.9 Aggiunta Requisito . . . . . 59
B.10 Aggiunta Versione . . . . . 59
B.11 Modifica File Versione . . . . . 60
B.12 Elenco ID Test Sequence . . . . . 60
B.13 Recupero Test Sequence per ID . . . . . 60
B.14 Recupero Test Sequence per Intervallo di ID . . . . . 61
B.15 Recupero Test Sequence per Nome . . . . . 61
B.16 Elenco Tutti i Test Sequence . . . . . 61
B.17 Elenco Hash delle Versioni . . . . . 61
B.18 Recupero Versione per Hash . . . . . 62
B.19 Download File per Hash . . . . . 62
B.20 Download File per Hash Multipli . . . . . 62
B.21 Recupero File per ID Test Sequence . . . . . 62
B.22 Recupero File per Intervallo di ID Test Sequence . . . . . 63
B.23 Recupero di Tutti i File . . . . . 63
B.24 Elenco Tutte le Versioni . . . . . 63
B.25 Elenco ID dei Feature Test Case . . . . . 64
B.26 Elenco Tutti i Feature Test Case . . . . . 64
B.27 Recupero Feature Test Case per ID . . . . . 64
B.28 Recupero ID Feature Test Case da Feature e Test Case . . . . . 64
B.29 Elenco ID dei Requisiti . . . . . 65
B.30 Recupero Requisito per ID . . . . . 65
B.31 Elenco Tutti i Requisiti . . . . . 65
B.32 Elenco Tutte le Associazioni Test Sequence e Feature Case . . . . . 66
B.33 Recupero Associazione Test Sequence e Feature Case . . . . . 66
B.34 Recupero Associazioni Feature Case per ID Test Sequence . . . . . 66
B.35 Recupero Associazioni Test Sequence per ID Feature Case . . . . . 67
B.36 Elenco Tutte le Associazioni Feature Case e Requisito . . . . . 67
B.37 Recupero Associazione Feature Case e Requisito . . . . . 67
B.38 Recupero Associazioni Requisito per ID Feature Case . . . . . 68
B.39 Recupero Associazioni Feature Case per ID Requisito . . . . . 68
B.40 Aggiornamento Test Sequence . . . . . 68
B.41 Aggiornamento Versione . . . . . 69
B.42 Aggiornamento Associazione Test Sequence e Feature Test . . . . . 69
B.43 Aggiornamento Associazione Feature Test e Requisito . . . . . 69
B.44 Caricamento Dati Coverage . . . . . 69

# Elenco delle figure

1.1 Schema concettuale dell'architettura del server e delle interazioni dell'operatore. . . . . 8
1.2 Rappresentazione semplificata delle principali relazioni tra le entità gestite dal TestSequence Repository. . . . . 9

2.1 Rappresentazione grafica e schema relazionale delle tabelle dell'Artifact Repository. . . . . 14
2.2 Schema logico e relazionale completo del database del TestSequence Repository. . . . . 15
2.3 Diagramma architetturale a tre livelli del Back-end e flusso delle richieste HTTP. . . . . 17
2.4 Schema architetturale e flusso di comunicazione dei moduli Front-end secondo il pattern MVC. . . . . 19
2.5 Mappatura dettagliata dei componenti del sistema all'interno dell'architettura a tre tier. . . . . 21

# Elenco delle tabelle


3.1 Librerie principali del Back-end e relative motivazioni d'uso. . . . . 23

4.1 Librerie principali del Front-end e relative motivazioni d'uso. . . . 37
4.2 Componenti di interfaccia per il feedback visivo e la gestione degli errori. . . . . 45
4.3 Differenze funzionali e strutturali tra le due Dashboard. . . . . 45
4.4 Architettura e responsabilità dei componenti *Renderer*. . . . . 46
4.5 Componenti *Renderer* per l'aggiunta delle risorse (`rendererAdders/`). 46
4.6 Componenti *Renderer* per la visualizzazione dei dati remoti (`rendererViewers/`). 47
4.7 Panoramica dei moduli *Handler* e relative responsabilità. . . . . 49
4.8 Panoramica dei moduli di utilità (`Utils`) e relative funzionalità. . . 50

A.1 Dettagli chiamata API `/ar/dashboard`. . . . . 53
A.2 Dettagli chiamata API `/ar/add`. . . . . 53
A.3 Dettagli chiamata API `/ar/rename_repository`. . . . . 54
A.4 Dettagli chiamata API `/ar/get`. . . . . 54
A.5 Dettagli chiamata API `/ar/list_artifacts`. . . . . 54
A.6 Dettagli chiamata API `/ar/get_creation_date_artifact`. . . . . 55
A.7 Dettagli chiamata API `/ar/get_created_by_artifact`. . . . . 55
A.8 Dettagli chiamata API `/ar/list_files_in_artifact`. . . . . 55
A.9 Dettagli chiamata API `/ar/download_repo`. . . . . 55
A.10 Dettagli chiamata API `/ar/download_file`. . . . . 56
A.11 Dettagli chiamata API `/ar/download`. . . . . 56
A.12 Dettagli chiamata API `/ar/remove`. . . . . 56

B.1 Dettagli chiamata API `/ts/dashboard`. . . . . 57
B.2 Dettagli chiamata API `/ts/add_ts`. . . . . 57
B.3 Dettagli chiamata API `/ts/delete_ts`. . . . . 58
B.4 Dettagli chiamata API `/ts/delete_TsFt`. . . . . 58
B.5 Dettagli chiamata API `/ts/add_ftc`. . . . . 58
B.6 Dettagli chiamata API `/ts/add_requirement_featureCase`. . . . . 58
B.7 Dettagli chiamata API `/ts/delete_ft_requirement`. . . . . 59
B.8 Dettagli chiamata API `/ts/add_testSequence_featureCase`. . . . . 59
B.9 Dettagli chiamata API `/ts/add_reqs`. . . . . 59
B.10 Dettagli chiamata API `/ts/add_version`. . . . . 59

B.11 Dettagli chiamata API /ts/edit_file. . . . . 60
B.12 Dettagli chiamata API /ts/get_test_sequence_ids. . . . . 60
B.13 Dettagli chiamata API /ts/get_test_sequence_by_id. . . . . 60
B.14 Dettagli chiamata API /ts/get_test_sequence_by_id_range. . . . . 61
B.15 Dettagli chiamata API /ts/get_test_sequence_by_name. . . . . 61
B.16 Dettagli chiamata API /ts/get_all_test_sequence. . . . . 61
B.17 Dettagli chiamata API /ts/get_version_hashes. . . . . 61
B.18 Dettagli chiamata API /ts/get_version_by_hash. . . . . 62
B.19 Dettagli chiamata API /ts/get_file_by_hash. . . . . 62
B.20 Dettagli chiamata API /ts/get_file_by_hashes. . . . . 62
B.21 Dettagli chiamata API /ts/get_files_by_ts_id. . . . . 62
B.22 Dettagli chiamata API /ts/get_files_by_ts_id_range. . . . . 63
B.23 Dettagli chiamata API /ts/get_all_files. . . . . 63
B.24 Dettagli chiamata API /ts/get_all_versions. . . . . 63
B.25 Dettagli chiamata API /ts/get_all_feature_test_case_ids. . . . . 64
B.26 Dettagli chiamata API /ts/get_all_feature_test_case. . . . . 64
B.27 Dettagli chiamata API /ts/get_feature_test_case_by_id. . . . . 64
B.28 Dettagli chiamata API /ts/get_id_feature_test_case_by_feature_-
and_test_case. . . . . 64
B.29 Dettagli chiamata API /ts/get_all_requirement_ids. . . . . 65
B.30 Dettagli chiamata API /ts/get_requirement_by_id. . . . . 65
B.31 Dettagli chiamata API /ts/get_all_requirements. . . . . 65
B.32 Dettagli chiamata API /ts/get_all_test_sequence_feature_test_case. 66
B.33 Dettagli chiamata API /ts/get_test_sequence_feature_test_case. . 66
B.34 Dettagli chiamata API /ts/get_test_sequence_feature_test_case_-
by_ts_id. . . . . 66
B.35 Dettagli chiamata API /ts/get_test_sequence_feature_test_case_-
by_ft_id. . . . . 67
B.36 Dettagli chiamata API /ts/get_all_feature_test_case_requirement. 67
B.37 Dettagli chiamata API /ts/get_feature_test_case_requirement. . . 67
B.38 Dettagli chiamata API /ts/get_feature_test_case_requirement_by_-
ft_id. . . . . 68
B.39 Dettagli chiamata API /ts/get_feature_test_case_requirement_by_-
r_id. . . . . 68
B.40 Dettagli chiamata API /ts/update_ts. . . . . 68
B.41 Dettagli chiamata API /ts/update_version. . . . . 69
B.42 Dettagli chiamata API /ts/update_testSequenceFeatureTest. . . . . 69
B.43 Dettagli chiamata API /ts/update_featureTestCaseRequirement. . . 69
B.44 Dettagli chiamata API /ts/add_coverage. . . . . 69

# Introduzione

Nell'ambito dei sistemi cyber-fisici, la continua evoluzione del software di controllo, le cui funzioni sono spesso critiche per la sicurezza degli utenti, rende necessario effettuare test di certificazione sempre più stringenti.

La natura critica di questi test, spesso caratterizzati da elevata complessità, rende difficile la loro esecuzione all'interno di ambienti automatizzati, come i sistemi di *Continuous Integration* (CI).

Nello specifico, questo progetto di stage si concentra sulla gestione di artefatti (*artifact)* e sequenze di test (*test sequence*) all'interno del sistema di CI, nell'ambito di un progetto più ampio dedicato allo sviluppo di software safety-critical per il controllo di motrici ferroviarie, svolto in collaborazione con un importante partner industriale italiano.

L'obiettivo del progetto di stage è dunque sviluppare una soluzione software per la gestione di tali dati che ne consenta l'acquisizione, la memorizzazione, il recupero e l'aggiornamento attraverso un'API REST e un'interfaccia grafica per facilitarne l'accesso, la visualizzazione e la modifica.

La presente relazione è strutturata come segue:

- **Capitolo 1:** descrizione dell'analisi del problema e dei requisiti da soddisfare, specificando i dati da gestire e le principali funzionalità richieste;
- **Capitolo 2:** illustrazione dell'architettura generale del sistema e del relativo flusso operativo, con particolare attenzione alle modalità di interazione tra i vari componenti;
- **Capitolo 3:** analisi dettagliata dell'implementazione del Back-End, degli standard seguiti e della relativa struttura, delineando come sono stati implementati i componenti chiave e le soluzioni adottate per superare i problemi riscontrati;
- **Capitolo 4:** analisi del Front-End, della sua struttura e delle modalità con cui esso comunica con il Back-End;
- **Capitolo 5:** presentazione delle conclusioni del progetto, tracciando un bilancio sia dal punto di vista formativo che dei risultati conseguiti, con un focus sui possibili sviluppi futuri.

In aggiunta, vengono fornite due appendici (**A** e **B**) dedicate alla documentazione e alla visualizzazione dettagliata delle API messe a disposizione dal Back-end.

# Glossario

Il Glossario raccoglie la definizione dei principali termini tecnici e acronimi utilizzati all'interno del documento, con lo scopo di garantire chiarezza e uniformità nella terminologia adottate.

**API RESTful:** *REpresentational State Transfer*. Architettura per interfacce di programmazione basata sul protocollo HTTP che consente l'interazione e l'integrazione programmatiche con sistemi terzi. Nel progetto, l'infrastruttura RESTful è sviluppata mediante **FastAPI**, un framework Python moderno ad alte prestazioni progettato per la gestione asincrona degli endpoint.

**Proprietà ACID e Gestione Transazionale:** *Atomicità, Coerenza, Isolamento, Durabilità*. Insieme di proprietà fondamentali che garantiscono che le transazioni in un database vengano elaborate in modo sicuro e affidabile durante l'esecuzione delle operazioni **CRUD** (*Create, Read, Update, Delete*). Tali garanzie permettono l'esecuzione del **Rollback**, ossia l'operazione di ripristino che annulla le modifiche effettuate da una transazione non completata per preservare l'integrità dei dati.

**Three-Tier Architecture:** Architettura a tre livelli. Modello architetturale software in cui le componenti funzionali del sistema sono suddivise in tre strati fisici o logici separati: Presentation Tier (interfaccia utente), Application Tier (logica di business) e Data Tier (persistenza dei dati).

**MVC:** *Model-View-Controller*. Pattern di progettazione software che separa l'applicazione in tre componenti interconnesse: il Model (gestione di dati e stato), la View (interfaccia e presentazione) e il Controller (gestione degli eventi e della logica di controllo).

**SSOT:** *Single Source of Truth*. Prassi architetturale che prevede di strutturare i dati dell'applicazione in modo che ogni elemento informativo sia memorizzato e modificabile esclusivamente in un unico punto dell'applicazione.

**ASGI:** *Asynchronous Server Gateway Interface*. Interfaccia standard per Python che consente ad applicazioni web asincrone (come FastAPI) di gestire un numero elevato di connessioni simultanee tramite protocollo HTTP.

**Race Condition:** Condizione critica che si verifica quando l'esito di un programma dipende dall'ordine imperfetto o imprevedibile con cui più thread o processi concorrenti accedono e modificano una risorsa condivisa.

**Starvation:** In concorrenza software, la situazione in cui a un thread o processo viene indefinitamente negato l'accesso a una risorsa condivisa a causa della priorità continuamente accordata ad altri processi.

# Capitolo 1

# Analisi del Problema

In questo capitolo viene analizzato nel dettaglio il problema riscontrato nell'infrastruttura esistente, individuando le esigenze operative del partner industriale, i dati coinvolti e la soluzione architetturale proposta.

## 1.1 Problema e Contesto

Il parner industriale necessita di validare i propri eseguibili in modo efficiente e tracciabile. L'infrastruttura preesistente presentava forti limitazioni nella gestione e nella scalabilità dei volumi di dati necessari alle procedure di test e verifica.

I dati gestiti dal sistema si dividono principalmente in due tipologie:

- **Artefatti:** rappresentano i file binari ed eseguibili risultanti dai processi di compilazione del codice sorgente. È stata espressamente richiesta una loro gestione generalizzata, al fine di consentire il deposito e l'organizzazione di diverse tipologie di file e cartelle.
- **Test Sequence:** sono l'insieme di file e parametri di test che devono essere applicati agli artefatti per verificare e approvare la correttezza del software ferroviario.

## 1.2 Requisiti del Sistema

### 1.2.1 Attori e Modalità di Interazione

Dal punto di vista operativo, gli attori che interagiscono con il sistema si dividono in due categorie principali:

- **Operatori Umani:** utilizzano l'interfaccia grafica accessibile tramite browser per navigare tra gli artefatti e le Test Sequence, nonché per modificare e caricare nuovi dati.

- **Sistemi Automatici:** interagiscono direttamente con il sistema attraverso le API REST, senza passare dall'interfaccia grafica, al fine di integrare le funzionalità della piattaforma con strumenti e pipeline esterne.

### 1.2.2 Requisiti Funzionali

I requisiti funzionali definiscono le funzionalità fondamentali che il sistema deve erogare per soddisfare le esigenze operative del partner industriale:

- **Gestione degli Artefatti:** il sistema deve consentire il caricamento, lo scaricamento, la modifica e la cancellazione di file e contenitori.
- **Gestione delle Test Sequence:** il sistema deve permettere l'importazione, la memorizzazione e la gestione delle versioni dei test e delle relative relazioni di tracciabilità.
- **Interfaccia Web (GUI):** il sistema deve mettere a disposizione un'interfaccia utente intuitiva per la visualizzazione, la ricerca e la gestione dei dati da parte degli operatori.
- **Esposizione di API RESTful:** il sistema deve rendere disponibili programmaticamente le funzionalità del sistema tramite API, consentendo l'integrazione con strumenti e pipeline esterne.

### 1.2.3 Requisiti Non Funzionali

I requisiti non funzionali definiscono le qualità e i vincoli operativi della soluzione:

- **Efficienza e Scalabilità:** il sistema deve essere in grado di gestire un'elevata mole di dati, garantendo prestazioni adeguate senza compromettere la scalabilità complessiva dell'infrastruttura.
- **Mantenibilità e Configurabilità:** l'architettura deve consentire l'abilitazione disgiunta dei moduli applicativi e la facile estensione delle funzionalità.

## 1.3 Soluzione Proposta

La soluzione individuata consiste nello sviluppo di un set di API RESTful attraverso python fastapi [1] per la gestione centralizzata ed efficiente di entrambe le tipologie di dati, affiancato da un'interfaccia grafica (GUI) destinata agli operatori.

Dal punto di vista architetturale, la soluzione si articola su due moduli concettuali:

- **Artifact Repository:** responsabile della conservazione e gestione degli artefatti.

- **TestSequence Repository:** responsabile dell'organizzazione dei test e delle relative sequenze di esecuzione.

Sebbene Artifact Repository e TestSequence Repository rappresentino due domini funzionali distinti, entrambi richiedono funzionalità comuni, quali la gestione dei file sul file system, la persistenza dei relativi metadati e l'esposizione delle funzionalità tramite API REST. È stato pertanto scelto di implementare le due componenti all'interno di un unico server applicativo, mantenendone separata la logica a livello modulare.

Tale scelta consente di ridurre la duplicazione del codice e semplificare le attività di deployment e manutenzione, preservando al contempo la possibilità di abilitare o disabilitare selettivamente ciascun repository mediante configurazione.

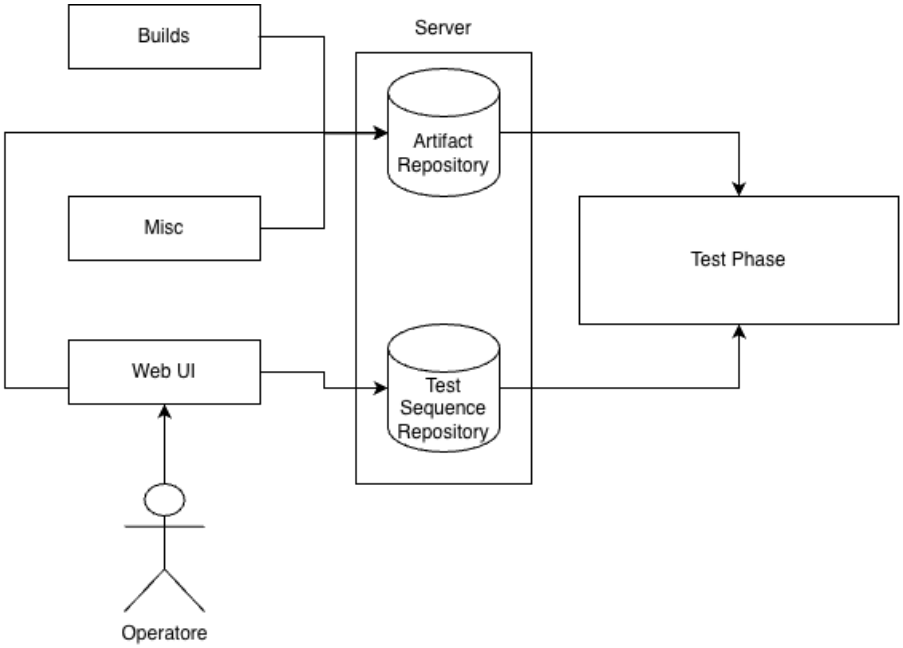



Figura 1.1: Schema concettuale dell'architettura del server e delle interazioni dell'operatore.

### 1.3.1 Artifact Repository

L'Artifact Repository adotta un'organizzazione dello spazio di archiviazione a *struttura monolivello*. L'albero di navigazione è pertanto limitato a un unico livello di profondità: sono ammessi esclusivamente contenitori di primo livello, all'interno dei quali possono essere memorizzati i file, mentre non è consentita la creazione di ulteriori sotto-cartelle.

Tale struttura permette il caricamento, la modifica, l'organizzazione e il download di qualsiasi tipologia di file all'interno dei contenitori di primo livello.

I file vengono memorizzati fisicamente sul file system del server, mentre un database SQL viene impiegato per indicizzare le risorse e gestire in modo rapido ed efficiente i relativi metadati.

Si è scelto esplicitamente di non memorizzare i file binari direttamente all'interno del database SQL. Il database viene infatti utilizzato per la gestione dei metadati e delle relazioni tra le risorse, mentre i contenuti effettivi dei file vengono mantenuti sul file system. Tale separazione evita di appesantire il database con la gestione dei dati binari, consentendo di demandare al file system la conservazione dei contenuti e al database la gestione delle informazioni necessarie alla loro indicizzazione e reperibilità.

### 1.3.2 TestSequence Repository

Il TestSequence Repository ha il compito di gestire le suite di test e le relative dipendenze. L'inserimento e l'aggiornamento dei test avvengono tramite l'acquisizione di uno specifico insieme di file di input via API:

- **Archivio ZIP:** contiene molteplici file Excel che costituiscono le versioni dei test eseguiti sugli artefatti.
- `TestSequenceNames.csv`: definisce i nomi identificativi delle Test Sequence, che vengono successivamente collegate alle rispettive versioni nel database.
- `FeaturesTestCases.csv`: specifica le associazioni tra le singole *feature* di sistema e i relativi *test case*.
- `Coverage.csv`: definisce la matrice di tracciabilità, correlando ciascuna Test Sequence ai rispettivi *feature test case*.
- `Requirements.csv`: definisce la copertura dei requisiti di progetto, specificando per ogni *test case* le specifiche funzionali che esso soddisfa.

Le relazioni tra le diverse entità coinvolte possono essere rappresentate mediante una visione semplificata del modello dei dati, utile a evidenziare i principali collegamenti tra versioni delle Test Sequence, Test Sequence, feature, test case e requisiti.

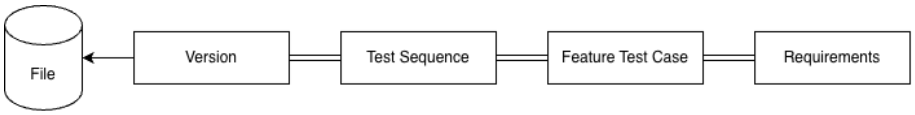



Figura 1.2: Rappresentazione semplificata delle principali relazioni tra le entità gestite dal TestSequence Repository.

Come mostrato nella Figura 1.2, il modello evidenzia le principali relazioni di tracciabilità che saranno successivamente rappresentate all'interno del database relazionale. La rappresentazione è volutamente semplificata e ha lo scopo di fornire una visione d'insieme delle connessioni tra le diverse entità, senza entrare

nel dettaglio della struttura fisica del database, che verrà analizzata nei capitoli successivi.

**Nota sull'evoluzione dei dati di input:** Durante l'analisi e l'implementazione del sistema è emerso che le informazioni contenute in `TestSequenceNames.csv` e `FeaturesTestCases.csv` sono integralmente derivabili dal file `Coverage.csv`, poiché quest'ultimo mappa formalmente tutte le relazioni tra Test Sequence e Feature Test Case. Nell'implementazione finale, tali file non costituiscono pertanto un input obbligatorio, in quanto le informazioni in essi contenute possono essere ricavate direttamente da `Coverage.csv`. Vengono comunque documentati in questa trattazione a fini di completezza e per descrivere l'evoluzione del formato dei dati utilizzato durante lo sviluppo e per una più semplice comprensione.

Tutte le relazioni e i metadati sopra descritti vengono persistiti all'interno di un database SQL organizzato in tabelle relazionali, mentre i contenuti effettivi dei file Excel e degli archivi vengono memorizzati sul file system del server.

## 1.4 Soluzioni alternative

Durante la fase iniziale del progetto di stage, sono state valutate diverse soluzioni architetturali e tecnologiche. Di seguito vengono discusse le opzioni che sono state scartate, corredate dalle relative motivazioni.

### 1.4.1 JFrog e Sonatype

Inizialmente si è preso in considerazione l'utilizzo di JFrog Artifactory [2] o di Sonatype Nexus Repository [3], con l'obiettivo di evitare di creare un sistema per la gestione asincrona dei file attraverso file system.

Tuttavia, dopo un'attenta analisi della documentazione di entrambe le piattaforme, è emerso che le rispettive versioni gratuite non consentivano il salvataggio di file in formato `.zip`, rendendole inadatte alle esigenze di archiviazione del progetto.

### 1.4.2 Flask

La prima versione del back-end prevedeva l'impiego del framework Flask [4] al posto di FastAPI [1].

Successivamente, un'analisi approfondita ha evidenziato che Flask gestisce le chiamate HTTP in modo sincrono. Questa limitazione avrebbe compromesso le prestazioni e la reattività richieste dal sistema nella gestione concorrente delle richieste asincrone, motivando così il passaggio a FastAPI.

### 1.4.3 Tabella relazionale unica

Nelle fasi preliminari di progettazione, lo schema del database prevedeva un'unica tabella relazionale per memorizzare sia i metadati dell'Artifact Repository sia le informazioni della Test Sequence Repository, servite inoltre da un'interfaccia web condivisa.

Con il progredire dell'analisi dei requisiti e una maggiore comprensione del dominio applicativo, è stato compreso che la Test Sequence Repository avrebbe gestito una mole di dati nettamente superiore. Per garantire una migliore separazione delle responsabilità, scalabilità e manutenibilità, si è quindi optato per la separazione logica e strutturale delle due repository.

# Capitolo 2

# Architettura e Pipeline

In questo capitolo viene analizzata nel dettaglio l'architettura complessiva del sistema, illustrando le interazioni e i flussi di comunicazione tra i principali componenti software.

## 2.1 Panoramica Generale

Come introdotto nel capitolo precedente, il progetto si articola su due domini applicativi principali:

- **Gestione degli Artefatti:** modulo destinato al deposito e all'organizzazione dei file binari ed eseguibili;
- **Gestione delle Test Sequence:** modulo dedicato alla memorizzazione e al tracciamento delle suite di test e dei relativi requisiti.

Al fine di soddisfare i requisiti operativi descritti, il sistema si compone delle seguenti architetture:

- **Interfaccia Web (Front-end):** un'applicazione grafica destinata agli operatori umani per la visualizzazione, la navigazione e la modifica manuale dei dati;
- **Server Applicativo (Back-end):** un set di servizi esposti tramite API RESTful per gestire la logica di business, l'accesso al database e le operazioni di I/O sul file system;

Come stabilito, entrambi i moduli (Artefatti e Test Sequence) risiedono all'interno del medesimo server applicativo per ottimizzare la manutenzione e la condivisione delle risorse. Nei paragrafi seguenti verrà inizialmente approfondita la struttura del Back-end, per poi analizzare l'architettura del Front-end e l'integrazione delle pipeline.

## 2.2 Back-end

Il Back-end costituisce il core dell'applicazione e ha il compito di svolgere le seguenti macro-attività:

- **Gestione della Persistenza:** interfacciare l'applicazione con un database relazionale SQL per l'indicizzazione dei metadati e delle relazioni di tracciabilità;
- **Gestione delle Richieste Asincrone:** elaborare le chiamate HTTP/REST in modo efficiente, sicuro e ad alte prestazioni;
- **Gestione del File System:** eseguire in modo atomico ed efficiente le operazioni di lettura e scrittura dei file memorizzati su disco.

### 2.2.1 Tecnologie Utilizzate

Per lo sviluppo del Back-end è stato adottato il linguaggio di programmazione Python (versione 3.10)[5]. Tale scelta è stata dettata dalla spiccata versatilità del linguaggio, dalla solida familiarità maturata con esso e dall'assenza di vincoli tecnologici stringenti da parte dell'azienda.

Per la gestione dei servizi web e l'esposizione delle API RESTful è stato scelto il framework `FastAPI` [1]. Questa soluzione garantisce elevate prestazioni grazie al supporto nativo alla programmazione asincrona.

### 2.2.2 Architettura del Database

Per la gestione della persistenza dei dati è stato adottato un sistema di gestione di database relazionale basato sul linguaggio SQL. La scelta di un modello relazionale è stata dettata dalla necessità di garantire elevata scalabilità, rapidità nelle operazioni di interrogazione e, soprattutto, un rigoroso rispetto dell'integrità dei dati.

Le informazioni gestite dal sistema richiedono il rispetto stringente dei vincoli di integrità referenziale e di dominio definiti nello schema. A tal fine, il database assicura il rispetto delle proprietà *ACID* [6]: in particolare, l'esecuzione delle operazioni all'interno di transazioni garantisce il meccanismo di *rollback* automatico qualora un'operazione violi uno dei vincoli imposti, ripristinando lo stato del sistema a una condizione coerente.

#### Database Artefatti

Il database dedicato all'Artifact Repository ha il compito di gestire i file organizzandoli secondo la struttura monolivello descritta in precedenza. Lo spazio di archiviazione è strutturato tramite contenitori di primo livello, denominati *repository*, all'interno dei quali possono essere memorizzati zero o più file.

Da tale modello concettuale si evince la necessità di definire due tabelle relazionali principali:

- `artifact_repository`: contiene la chiave primaria identificativa (`id`), il nome del contenitore (`name`) e la data di creazione (`created_at`);
- `artifact_file`: contiene l'identificativo univoco del file (`id`), il riferimento alla repository di appartenenza tramite chiave esterna (`artifact_id`), il nome del file (`name`), l'utente che ha effettuato il caricamento (`created_by`) e la relativa data di creazione (`created_at`).

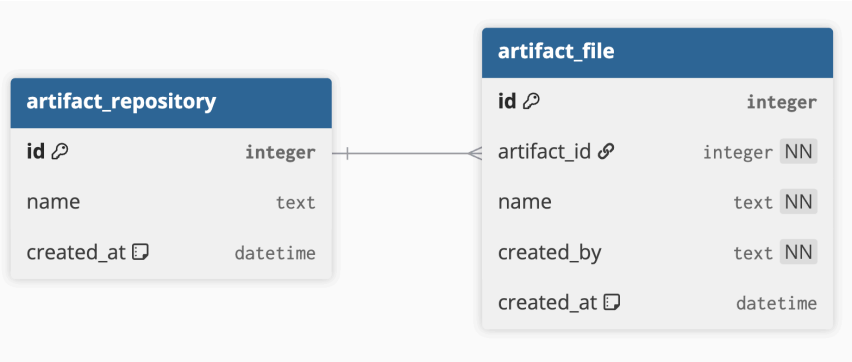



Figura 2.1: Rappresentazione grafica e schema relazionale delle tabelle dell'Artifact Repository.

### Database Test Sequence

Il database della TestSequence Repository è progettato per gestire le suite di test e modellare le complesse relazioni di tracciabilità tra le modalità di esecuzione e i requisiti di progetto.

Nel processo di validazione di un artefatto software, quest'ultimo viene sottoposto all'esecuzione di una specifica *Test Sequence*: essa rappresenta uno scenario operativo composto da zero o più *Test Case*, ciascuno dei quali è associato ai rispettivi requisiti (*Requirement*) che deve soddisfare e verificare. Poiché la logica e i parametri operativi di ciascun test sono definiti all'interno di file Excel soggetti a modifiche e aggiornamenti nel tempo, si è introdotto il concetto di *Version*, la quale traccia la specifica revisione del file associato alla Test Sequence.

Sulla base di tale impianto concettuale, lo schema del database si articola nelle seguenti tabelle relazionali:

- `test_sequence`: rappresenta l'entità concettuale della Test Sequence. Memorizza i metadati di alto livello come la denominazione (`name`), il sottosistema di appartenenza (`subsystem`) e la tipologia di dati a cui fa riferimento (`train_data_set`). Mantiene inoltre il riferimento diretto all'ultima revisione pubblicata (`latest`) tramite una chiave esterna verso l'hash della tabella `version`. Le sequenze di test derivano dalla documentazione ufficiale dell'A-

genzia dell'Unione Europea per le Ferrovie, con particolare riferimento alla specifica *Subset-076* [7].

- `version`: traccia lo storico e l'evoluzione temporale dei file di test. Invece di utilizzare un id progressivo, impiega come chiave primaria l'hash. Mantiene il vincolo con la Test Sequence padre (`ts_id`), il nome del file sul file system (`filename`), i metadati di tracciabilità (`created_on`, `created_by`, `notes`) e il puntatore alla versione precedente (`previous_version`) per consentire la ricostruzione della cronologia delle modifiche;
- `feature_test_case`: modella la coppia univoca composta da una funzionalità di sistema (`feature`) e il corrispondente caso di test (`test_case`). Il suo riconoscimento è dato da un `id` unico.
- `test_sequence_feature_test_case`: tabella che realizza la relazione molti-a-molti tra le Test Sequence (`ts_id`) e le coppie Feature/Test Case (`ft_id`).
- `feature_test_case_requirement`: tabella che mappa la relazione molti-a-molti tra le coppie Feature/Test Case (`ft_id`) e i requisiti funzionali (`r_id`).
- `requirement`: costituisce il catalogo dei requisiti di sistema e memorizza unicamente l'identificativo univoco (`id`) e la denominazione del requisito (`name`);
- `requirement_tag`: definisce la relazione uno-a-molti tra i requisiti e i relativi tag di classificazione, mediante l'associazione dell'identificativo del requisito (`requirement_id`) a ogni singola etichetta (`tag`).

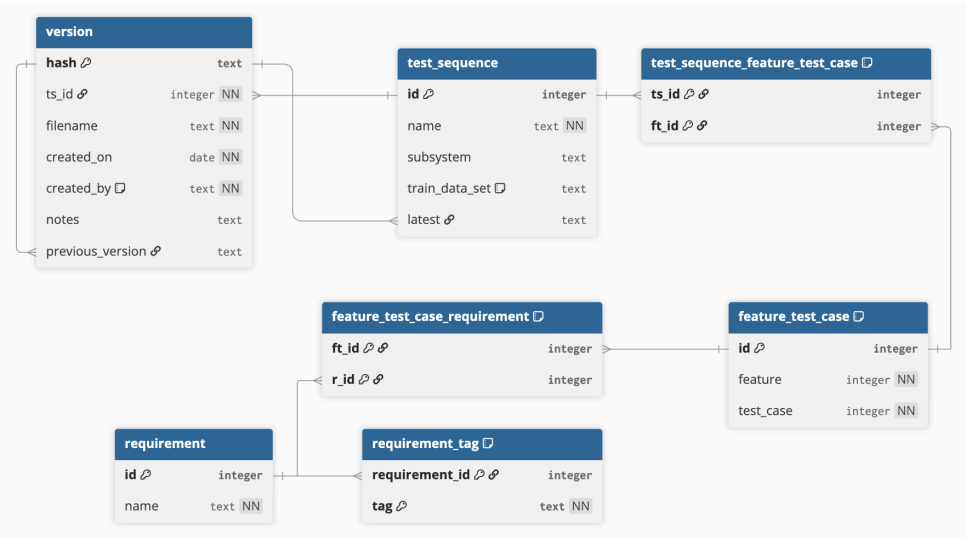



Figura 2.2: Schema logico e relazionale completo del database del TestSequence Repository.

### 2.2.3 Architettura del Back-end

Il Back-end, come anticipato, è sviluppato in Python e sfrutta il framework *FastAPI*[1] per l'esposizione e la gestione delle API RESTful. Per descrivere l'architettura adottata in modo trasparente, si prescinde momentaneamente dall'interfaccia utente, analizzando il flusso di elaborazione a partire da una generica chiamata API inviata direttamente al server.

L'architettura del Back-end è organizzata secondo un modello layered a tre livelli a responsabilità separata:

- **`main.py` (Livello di Routing e API):** rappresenta il punto di ingresso dell'applicazione. Sfruttando le funzionalità di routing di *FastAPI*[1], definisce gli endpoint e intercetta le richieste HTTP in ingresso. In base al percorso richiesto, `main.py` smista le chiamate verso la componente logica appropriata (`TestSequenceRepository` o `ArtifactRepository`). Alla ricezione dell'esito dell'operazione, si occupa di formattare la risposta HTTP, impostando gli opportuni codici di stato (*Status Code*) e gestendo l'eventuale restituzione di eccezioni o errori.

- **`ArtifactRepository.py`** e **`TestSequenceRepository.py`** **(Livello Logico/Applicativo):** costituiscono il livello intermedio dell'architettura e concentrano la logica di business relativa, rispettivamente, agli artefatti e alle Test Sequence. Ricevono i dati dal livello API, validano le regole applicative e orchestrano le operazioni necessarie delegando l'accesso fisico alle risorse ai componenti del livello sottostante. Una volta raccolti i risultati, effettuano le opportune trasformazioni prima di restituirli a `main.py`.

- **`FileManager.py`** / **`SQLManager.py`** **(Livello di Persistenza):** rappresentano il livello di accesso alle risorse fisiche e alla memoria di massa. Nello specifico, `FileManager` si occupa dell'I/O direttamente sul file system del server (salvataggio, eliminazione e lettura dei file Excel o binari), mentre `SQLManager` incapsula la gestione del database relazionale, eseguendo le query SQL e gestendo le transazioni.

La rigida separazione delle responsabilità tra i tre livelli permette di disaccoppiare la gestione del protocollo HTTP dalla logica di business e dalle specifiche implementative della persistenza, garantendo elevata modularità, riusabilità del codice e facilità di manutenzione.

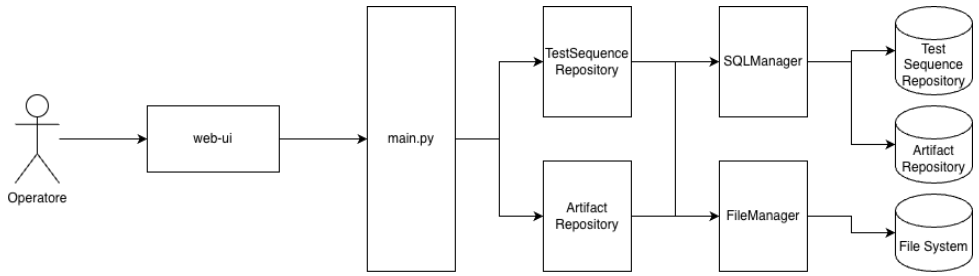



Figura 2.3: Diagramma architetturale a tre livelli del Back-end e flusso delle richieste HTTP.

## 2.3 Front-end

L'interfaccia utente (Front-end) ha l'obiettivo di fornire un'esperienza d'uso intuitiva ed efficiente, consentendo agli operatori di gestire l'intero ciclo di vita dei dati (caricamento, consultazione, modifica ed eliminazione) senza che sia necessaria una conoscenza approfondita dell'architettura interna del sistema.

Per rispondere ai requisiti operativi del progetto, sono state realizzate due *dashboard* distinte e specializzate:

- **Artifact Dashboard:** dedicata alla gestione, tracciabilità e visualizzazione degli artefatti software;
- **Test Sequence Dashboard:** progettata per la consultazione e il controllo delle Test Sequence e correlati.

Nel dettaglio, il Front-end si articola attorno al soddisfacimento delle seguenti macro-funzionalità:

- **Caricamento e ingestione dei dati:** gestione dell'upload dei file tramite interfaccia web;
- **Validazione e gestione degli errori:** verifica preventiva dei dati inseriti e segnalazione puntuale all'utente delle eventuali anomalie riscontrate, specificando la natura e la posizione esatta dell'errore;
- **Visualizzazione ed esplorazione dei dati:** organizzazione delle informazioni in tabelle interattive e visualizzazioni dettagliate;
- **Modifica e aggiornamento:** strumenti per la gestione delle revisioni dei dati persistiti;
- **Eliminazione delle risorse:** funzionalità di rimozione in sicurezza dei dati;
- **Verifica e collaudo integrato delle API:** strumenti d'interfaccia dedicati per eseguire chiamate di prova verso specifici endpoint, fornendo la possibilità di scegliere parametri mirati al fine di verificare direttamente il corretto funzionamento delle API.

### 2.3.1 Tecnologie utilizzate

Per lo sviluppo dell'interfaccia si è preferito un approccio essenziale e leggero, evitando l'uso di framework complessi come React, Angular o Vue per non appesantire il progetto. Tale scelta è stata dettata dalla volontà di evitare un'inutile sovra-complicazione architetturale, mantenendo il codice snello, di facile manutenzione e focalizzato sullo scopo primario del sistema: la gestione e la consultazione rapida delle due dashboard.

Le tecnologie adottate per lo sviluppo del Front-end sono nello specifico:

- **HTML5 e CSS3:** impiegati per la definizione della struttura semantica delle pagine e per lo stile grafico di base.
- **Bootstrap 5:** utilizzato come framework CSS di riferimento per garantire un'interfaccia moderna, pulita e visivamente curata.
- **jQuery e JavaScript (ES6):** impiegati per la gestione delle interazioni dinamiche all'interno dell'interfaccia. In particolare, la libreria jQuery è stata integrata per semplificare la manipolazione del Document Object Model (DOM).

### 2.3.2 Architettura del Front-end

Il Front-end è stato progettato seguendo il pattern architetturale *Model-View-Controller* (MVC) al fine di garantire una chiara separazione delle responsabilità, modularità e manutenibilità del codice. Data la forte analogia strutturale e comportamentale tra le due dashboard, l'organizzazione architetturale illustrata di seguito si applica in modo speculare sia all'interfaccia dell'*Artifact Repository* sia a quella del *Test Sequence Repository*.

**Model (Gestione Dati e Stato):** gestisce la logica dei dati, la comunicazione con i servizi remoti e lo stato dell'applicazione client-side.

- `state/store.js`: funge da *Single Source of Truth*, gestendo lo stato globale e reattivo dell'applicazione nel client.
- Moduli nella directory `/api` (`ArtifactApi.js`, `client.js`, ecc.): astraggono l'accesso ai dati, gestiscono la logica di comunicazione con il Back-end ed eseguono le richieste HTTP RESTful.

**View (Interfaccia Grafica e Presentazione):** definisce gli elementi visivi e la struttura presentazionale dell'interfaccia.

- Pagine HTML (`dashboardArtifacts.html`, `dashboardTest.html`): definiscono la struttura statica del DOM, il layout dei contenitori e le finestre di dialogo (*modal*).
- Componenti nella directory `/components` (`rendererFileSystem.js`, `logger.js`, ecc.): moduli responsabili della generazione e del rendering di-

namico dell'interfaccia grafica a partire dallo stato corrente contenuto nel Model.

**Controller (Gestione Eventi e Logica di Controllo):** gestisce il flusso applicativo, intercetta le interazioni dell'utente e coordina la comunicazione tra Model e View.

- Entry Point (`mainArtifact.js`, `mainTest.js`): agiscono da orchestratori principali, gestendo il caricamento del DOM e l'inizializzazione dei vari moduli.
- Handler nella directory `/handlers` (`buttonHandlersArtifact.js`, ecc.): intercettano l'input e gli eventi generati dall'utente (click, invio di form, selezioni), invocano gli aggiornamenti dello stato nel Model e richiedono il conseguente *re-rendering* della View.

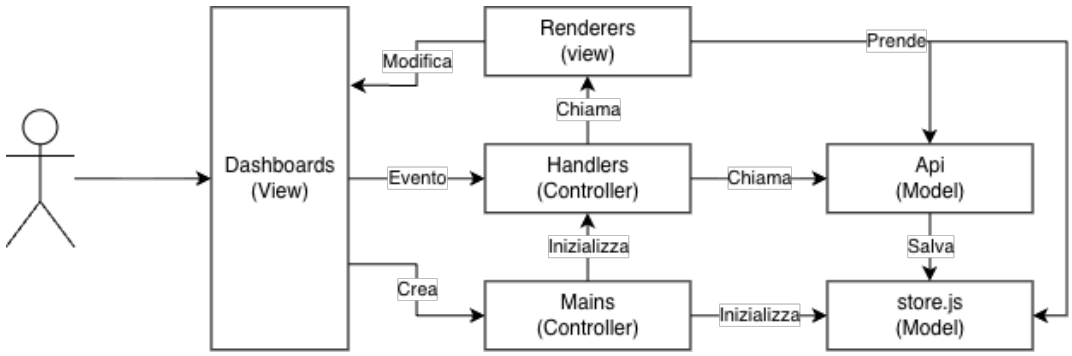



Figura 2.4: Schema architetturale e flusso di comunicazione dei moduli Front-end secondo il pattern MVC.

## 2.4 Architettura a Tre Tier

Dopo aver analizzato i singoli moduli, è possibile dimostrare come la struttura dell'intero sistema segua un'architettura a tre tier, identificando chiaramente i componenti appartenenti a ciascuno strato e le relative modalità di interazione:

1. **Presentation Tier (Front-end):** Risiede sul client dell'utente ed è costituito dalle pagine HTML5, dallo stile Bootstrap 5[8] e dalla logica di gestione dello stato in JavaScript. Si occupa esclusivamente dell'interfaccia grafica e di intercettare le azioni dell'operatore.
2. **Application Tier (Back-end):** Risiede sul server ed è implementato in Python con il framework `FastAPI`. Include il coordinatore principale (`main.py`) e i moduli (`ArtifactRepository.py` e `TestSequenceRepository.py`). Riceve le richieste dal Presentation Tier, applica la logica di dominio e coordina l'accesso ai dati senza gestire direttamente la persistenza fisica.

3. **Data Tier (Persistenza):** È costituito dai componenti di basso livello `SQLManager.py` e `FileManager.py`, i quali si interfacciano rispettivamente con i dati persistenti.

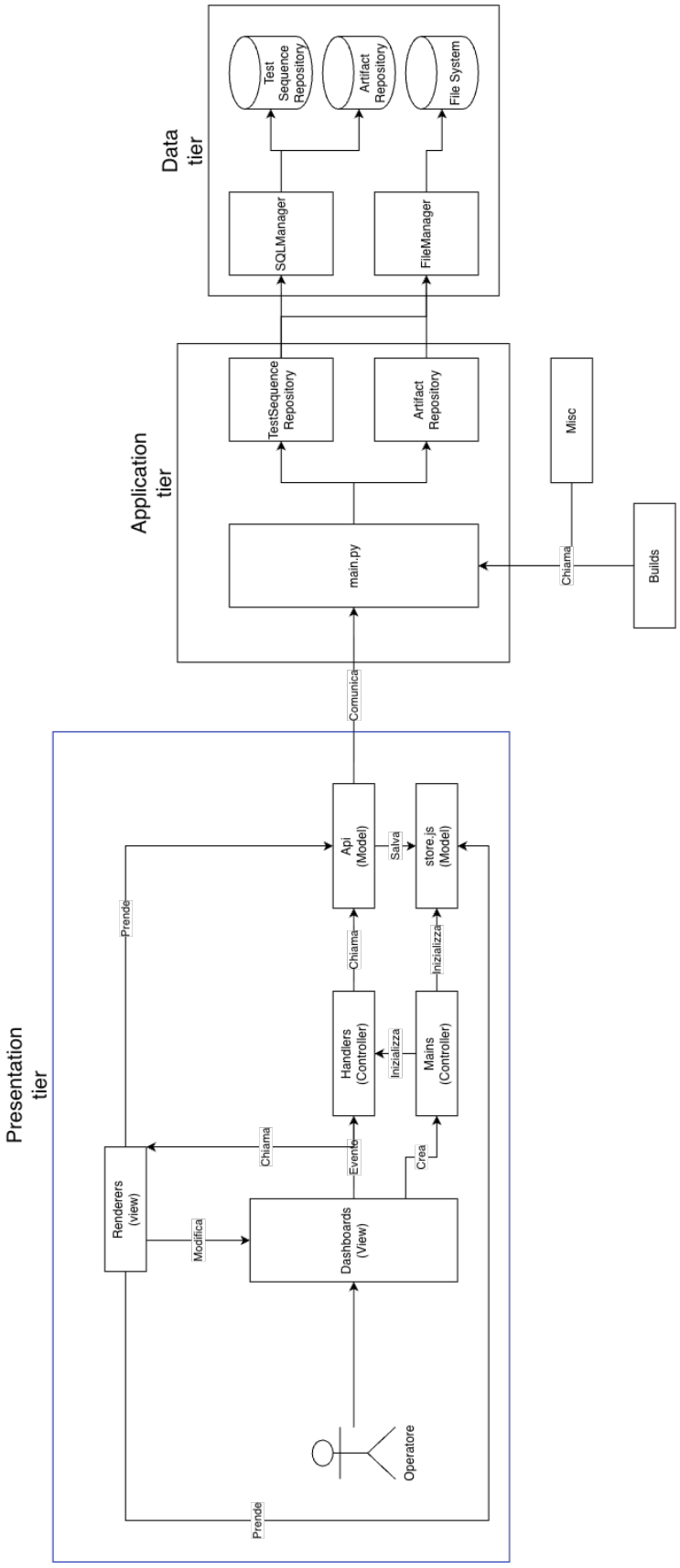



Figura 2.5: Mappatura dettagliata dei componenti del sistema all'interno dell'architettura a tre tier.

# Capitolo 3

# Back-end

Il livello di Back-end ha la responsabilità di gestire l'esposizione delle API RESTful, l'elaborazione della logica applicativa e la persistenza dei dati. In questo capitolo vengono analizzate nel dettaglio le scelte architetturali, i pattern progettuali e le tecnologie adottate per lo sviluppo del lato server.

## 3.1 Librerie e Dipendenze

Le dipendenze del progetto sono formalizzate all'interno del file di configurazione `requirements.txt`. Per garantire la corretta esecuzione del sistema, è richiesta l'installazione di tutti i pacchetti indicati, raccomandando l'uso di un ambiente virtuale Python[5] e del gestore di pacchetti `pip` tramite il comando:

```
pip install -r requirements.txt
```

La Tabella 3.1 riassume le librerie principali utilizzate nel Back-end, specificandone la funzione e la motivazione d'uso nell'architettura.

## 3.2 Chiamate API

La gestione e l'instradamento delle chiamate API RESTful sono centralizzati all'interno del modulo `main.py`. In quanto punto di ingresso (*entry point*) dell'applicazione server-side, questo file ha la responsabilità di orchestrare l'avvio del sistema, istanziando i componenti del livello logico sottostante—ovvero `ArtifactRepository` e `TestSequenceRepository`—e configurando le risorse necessarie al loro funzionamento.

L'esecuzione diretta del file (tramite il comando `python main.py`) avvia il ciclo di vita del server applicativo eseguendo due operazioni fondamentali:

1. **Inizializzazione dell'applicazione:** viene istanziata la classe di coordinamento `Coordinator` (`coordinator = Coordinator()`), la quale configura l'i-

Tabella 3.1: Librerie principali del Back-end e relative motivazioni d'uso.

| **Nome Libreria** | **Funzione** | **Motivazione dell'utilizzo** |
|---|---|---|
| `fastapi` | Framework Web API | Permette la creazione di endpoint RESTful ad alte prestazioni, con supporto nativo all'asincronia e validazione automatica dei dati. |
| `uvicorn` | Server ASGI | Consente l'esecuzione asincrona del server applicativo FastAPI. |
| `readerwriterlock` | Controllo concorrenza | Implementa i meccanismi di sincronizzazione Lettori/Scrittori per evitare *race condition* sull'accesso al File System. |
| `python-multipart` | Gestione dati FORM | Abilita il parsing e lo streaming efficiente di file e dati inviati tramite multipart/form-data. |

stanza dell'applicazione web FastAPI resa accessibile mediante l'attributo `coordinator.app`.

2. **Esecuzione del server:** attraverso il server ASGI `Uvicorn`[9], viene avviato l'ascolto delle richieste HTTP sull'applicazione istanziata tramite la direttiva:

```
uvicorn.run("main:app", host="127.0.0.1", port=8000, reload=True)
```

Questa configurazione imposta l'interfaccia di *loopback* locale (`127.0.0.1`) sulla porta `8000`, attivando la modalità di *hot reload* per il ricaricamento automatico del server a fronte di modifiche al codice sorgente.

### 3.2.1 Inizializzazione del Servizio

Durante la fase di avvio, il costruttore della classe `Coordinator` (`__init__`) si occupa di istanziare e configurare tutte le risorse e i servizi necessari al funzionamento del sistema back-end attraverso una sequenza logica ben definita:

1. **Istanziazione dell'applicazione web e asset statici:** viene creata l'istanza principale dell'applicazione tramite la libreria `FastAPI` (`self.app = FastAPI()`). Successivamente, viene montata la directory `static/` come cartella statica del server; questo passaggio è fondamentale per servire direttamente al browser i file del front-end (HTML, CSS, JavaScript) all'accesso alla dashboard.

2. **Istanziazione dei gestori di basso livello e dei repository:** vengono creati i componenti di persistenza `FileManager` e `SQLManager`. Successi-

vamente, questi manager vengono iniettati come dipendenze all'interno dei repository di dominio, secondo lo schema:

```
self.ts_repo = TestSequenceRepository(
    self.config_test, self.manager_sql, self.manager_file
)
```

La stessa logica viene applicata per l'inizializzazione di `ArtifactRepository`, garantendo il principio di separazione delle responsabilità e il disaccoppiamento dei componenti.

3. **Registrazione delle rotte API:** viene invocata internamente la funzione `_register_routes()`, la quale associa le rotte definite nell'applicazione FastAPI ai rispettivi metodi di logica applicativa esposti dai repository.

### 3.2.2 Standardizzazione dei Valori di Ritorno

Per garantire consistenza e prevedibilità nell'interazione con i client, tutte le API esposte dal server adottano uno schema di risposta unificato. Ogni chiamata HTTP restituisce un oggetto JSON strutturato composto da due campi principali:

- **`code`:** rappresenta lo stato dell'operazione, veicolando il codice di risposta HTTP (ad esempio `200 OK` in caso di successo, `400 Bad Request` o `500 Internal Server Error` in caso di errore);
- **`data`:** contiene il carico utile (*payload*) della risposta, la cui struttura varia dinamicamente in base all'esito della richiesta:
  - In caso di errore o eccezione, il campo racchiude una stringa descrittiva contenente il messaggio dettagliato dell'errore;
  - In caso di operazione completata con successo, il campo ospita i dati richiesti dal client (ad esempio oggetti JSON, valori booleani o array di elementi).

L'adozione di questo pattern standardizzato semplifica significativamente l'elaborazione delle risposte e la gestione centralizzata degli errori da parte dell'interfaccia front-end.

### 3.2.3 Standard API

Tutte le API esposte dal sistema seguono uno schema di denominazione rigoroso e omogeneo, strutturato per riflettere il modulo di appartenenza:

- **`/ts/*`:** prefisso riservato a tutti gli endpoint relativi alla gestione delle *Test Sequence*;

- **`/ar/*`:** prefisso riservato a tutti gli endpoint dedicati alla gestione degli *Artifact*.

Salvo diversamente specificato, tutti gli endpoint che utilizzano un metodo HTTP diverso da `GET` (ad esempio `POST`, `DELETE`) richiedono l'invio dei dati tramite parametri in formato **FORM** (`application/x-www-form-urlencoded` o `multipart/form-data`).

All'inizio dell'esecuzione di ciascuna funzione di rotta, viene applicato un meccanismo preliminare di guardia per verificare lo stato di abilitazione del rispettivo servizio. Attraverso il controllo dei parametri di configurazione (`config_artifact` e `config_test`), il server determina se la funzionalità richiesta è attiva. Se il modulo corrispondente risulta disabilitato, l'esecuzione viene interrotta immediatamente sollevando un'eccezione HTTP con codice di stato `404 Not Found`:

```
if not self.config_artifact["enabledArtifacts"]:
    raise HTTPException(status_code=404, detail="Not enabled")

if not self.config_test["enabledTests"]:
    raise HTTPException(status_code=404, detail="Not enabled")
```

Questo approccio garantisce una gestione centralizzata e sicura del ciclo di vita dei servizi, consentendo di attivare o disattivare dinamicamente intere sezioni dell'applicazione senza dover modificare la struttura delle rotte esposte.

Per la consultazione di dettaglio dei singoli endpoint, si rimanda all'Appendice A per le API relative agli Artifact e all'Appendice B per le API dedicate alle Test Sequence.

## 3.3 Artifact e TestSequence Repository

I moduli `ArtifactRepository` e `TestSequenceRepository` rappresentano il livello di astrazione intermedia (*Business Logic Layer*) che interpola le chiamate provenienti dagli endpoint API con i moduli di persistenza sottostanti.

### 3.3.1 Valori di Ritorno

I valori di ritorno delle funzioni seguono sempre una struttura standard composta dai seguenti campi:

- **code**: un valore intero che indica lo stato dell'operazione, mappato sui codici di stato dello standard HTTP;
- **data**: i dati restituiti in caso di successo (null o assente in caso di errore);
- **error**: il dettaglio dell'errore restituito in caso di fallimento (null o assente in caso di successo).

## 3.4 Gestione della Persistenza Relazionale

La gestione dei dati strutturati e dei metadati associati a file e sequenze è affidata al modulo `SQLManager`. Esso funge da astrazione per il database relazionale, isolando la composizione delle query SQL dal resto dell'applicazione.

### 3.4.1 Inizializzazione dello Schema Database

L'inizializzazione dello schema del database relazionale viene eseguita all'interno del costruttore della classe `SQLManager`. In questa fase viene istanziata la connessione al database SQLite tramite l'oggetto `self.conn`:

```
self.conn = sqlite3.connect(
    "rfi_coordinator.db",
    check_same_thread=False
)
```

Il parametro `check_same_thread` è impostato esplicitamente a `False` per disabilitare i controlli nativi di SQLite sul multi-threading, consentendo così la gestione di operazioni asincrone e concorrenti. Tale scelta delega l'onere della sincronizzazione e del controllo della concorrenza direttamente all'applicazione (le cui strategie implementative verranno descritte nel capitolo successivo).

Di seguito viene presentata la definizione delle tabelle relazionali:

```
-- Gestione Artifacts
CREATE TABLE IF NOT EXISTS artifact_repository (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS artifact_file (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artifact_id INTEGER NOT NULL,
    name TEXT NOT NULL,
    created_by TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (artifact_id) REFERENCES artifact_repository(id)
);

-- Gestione Sequenze di Test e Versionamento
CREATE TABLE IF NOT EXISTS test_sequence (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    subsystem TEXT,
    train_data_set TEXT CHECK (
        train_data_set IN (
```

```
            'generic', 'gamma1', 'gamma2',
            'lambda1', 'lambda2', 'lambda3'
        )
    ),
    latest TEXT,
    FOREIGN KEY (latest) REFERENCES version(hash)
);

CREATE TABLE IF NOT EXISTS version (
    hash TEXT PRIMARY KEY,
    ts_id INTEGER NOT NULL,
    filename TEXT NOT NULL,
    created_on DATE NOT NULL DEFAULT CURRENT_DATE,
    created_by TEXT NOT NULL DEFAULT 'Admin',
    notes TEXT,
    previous_version TEXT,
    FOREIGN KEY (ts_id) REFERENCES test_sequence(id),
    FOREIGN KEY (previous_version) REFERENCES version(hash)
);

-- Entita e Tabelle Associative (N:N)
CREATE TABLE IF NOT EXISTS feature_test_case (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    feature INTEGER NOT NULL,
    test_case INTEGER NOT NULL,
    UNIQUE(feature, test_case)
);

CREATE TABLE IF NOT EXISTS requirement (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS requirement_tag (
    requirement_id INTEGER,
    tag TEXT NOT NULL,
    PRIMARY KEY (requirement_id, tag),
    FOREIGN KEY (requirement_id) REFERENCES requirement(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS test_sequence_feature_test_case (
    ts_id INTEGER NOT NULL,
    ft_id INTEGER NOT NULL,
    PRIMARY KEY(ts_id, ft_id),
    FOREIGN KEY(ts_id) REFERENCES test_sequence(id),
    FOREIGN KEY(ft_id) REFERENCES feature_test_case(id)
);
```

```
CREATE TABLE IF NOT EXISTS feature_test_case_requirement (
    ft_id INTEGER NOT NULL,
    r_id INTEGER NOT NULL,
    PRIMARY KEY(ft_id, r_id),
    FOREIGN KEY(ft_id) REFERENCES feature_test_case(id),
    FOREIGN KEY(r_id) REFERENCES requirement(id)
);
```

Infine, per ottimizzare i tempi di esecuzione delle query di join e la navigazione delle relazioni più frequenti, sono stati definiti indici specifici sulle chiavi esterne e sui campi di ricerca principali:

```
CREATE INDEX IF NOT EXISTS idx_version_ts
    ON version(ts_id);

CREATE INDEX IF NOT EXISTS idx_version_previous
    ON version(previous_version);

CREATE INDEX IF NOT EXISTS idx_test_sequence_latest
    ON test_sequence(latest);

CREATE INDEX IF NOT EXISTS idx_feature_test_case_feature
    ON feature_test_case(feature);

CREATE INDEX IF NOT EXISTS idx_feature_test_case_test_case
    ON feature_test_case(test_case);
```

### 3.4.2 Gestione della Concorrenza e delle Race Condition

A causa dell'abilitazione delle operazioni asincrone e della contestuale disattivazione dei controlli nativi di SQLite sul multi-threading (tramite il parametro `check_same_thread=False`), la gestione della concorrenza e la prevenzione delle *race condition* sono state delegate interamente all'applicazione.

Per garantire la consistenza dei dati senza compromettere le prestazioni, è stato adottato il pattern classico per il problema dei lettori e scrittori (*Readers-Writers Problem*), implementato tramite la libreria Python `readerwriterlock`. In particolare, viene utilizzata la variante `RWLockFairD`, che garantisce la correttezza distribuendo l'accesso equamente tra operazioni di lettura e scrittura ed evitando situazioni di starvation:

```
from readerwriterlock import rwlock

# Inizializzazione del lock equo per lettori e scrittori
self.locker = rwlock.RWLockFairD()
```

La gestione delle sezioni critiche avviene sfruttando il contesto offerto dai *context manager* di Python (`with`):

- **Operazioni di sola lettura**: utilizzano un lock condiviso (*read-lock*), che consente a più coroutine o thread di accedere simultaneamente alle risorse in lettura:

```
with self.locker.gen_rlock():
    # Esecuzione di query SELECT o lettura dati
    pass
```

- **Operazioni di scrittura o modifica**: richiedono un lock esclusivo (*write-lock*), che blocca sia le letture sia altre scritture per tutta la durata della transazione, garantendo l'atomicità dell'operazione:

```
with self.locker.gen_wlock():
    # Esecuzione di INSERT, UPDATE, DELETE o transazioni di scrittura
    pass
```

### 3.4.3 Analisi Operativa per Casistica

Al fine di evitare la trattazione ridondante di ogni singola funzione dell'interfaccia, in questa sezione vengono analizzati i pattern architetturali e operativi principali che regolano l'interazione diretta tra il livello applicativo e il database tramite la classe `SQLManager`. La struttura delle operazioni si riconduce fondamentalmente alle quattro casistiche CRUD standard.

**Operazioni di Creazione (Create)** Nelle operazioni di inserimento, la sicurezza della transazione è garantita dall'acquisizione del lock di scrittura esclusivo (`gen_wlock()`). All'interno del blocco con contesto di transazione (`with self.conn`), viene eseguita una query parametrizzata per prevenire vulnerabilità di SQL Injection.

```
try:
    with self.locker.gen_wlock():
        with self.conn:
            self.conn.execute(
                """
                INSERT INTO version (hash, ts_id, created_by, notes,
                                     previous_version, filename)
                VALUES (?, ?, ?, ?, ?, ?)
                """,
                (
                    hashId,
                    ts_id,
                    created_by,
                    notes,
                    previous_version,
```

```
                    filename,
                ),
            )
            return {"success": True}
except Exception as e:
    return {"success": False, "data": str(e)}
```

**Operazioni di Aggiornamento (Update)** Per la modifica dei dati è stato implementato un pattern dinamico: per evitare la duplicazione del codice, la funzione accetta l'identificativo del record e un dizionario `changes` contenente le coppie chiave-valore corrispondenti ai campi da aggiornare. La query SQL viene generata dinamicamente assemblando i campi ricevuti:

```
def update_ts(self, id, changes):
    if not changes:
        return {"success": False, "data": "No changes provided"}
    try:
        with self.locker.gen_wlock():
            cursor = self.conn.execute(
                f"UPDATE test_sequence SET "
                f"{', '.join([f'{k} = ?' for k in changes.keys()])} "
                f"WHERE id = ?",
                [*changes.values(), id],
            )
            self.conn.commit()

            if cursor.rowcount == 0:
                return {
                    "success": False,
                    "data": f"No record found with id {id}",
                }

            return {
                "success": True,
                "data": f"Updated test_sequence ID {id} successfully",
            }

    except sqlite3.IntegrityError as e:
        self.conn.rollback()
        return {
            "success": False,
            "data": f"Database integrity error: {str(e)}"
        }

    except Exception as e:
        self.conn.rollback()
        return {
```

```
            "success": False,
            "data": f"Unexpected database error: {str(e)}",
        }
```

Questa struttura viene replicata per le diverse tabelle mantenendo funzioni dedicate. La scelta di separare i metodi di aggiornamento per ciascuna entità relazionale (anziché adottare un unico metodo generico) è stata guidata dalla volontà di facilitare l'estensione futura di logiche di aggiornamento e validazione specifiche per ciascun dominio.

**Operazioni di Eliminazione (Delete)** Analogamente alla creazione, le procedure di eliminazione acquisiscono il lock esclusivo di scrittura prima di eseguire l'istruzione `DELETE` parametrizzata:

```
def delete_ts(self, ts_id):
    try:
        with self.locker.gen_wlock():
            with self.conn:
                self.conn.execute(
                    """
                    DELETE FROM test_sequence
                    WHERE id = ?
                    """,
                    (ts_id,),
                )
                return {"success": True}
except Exception as e:
    return {"success": False, "data": str(e)}
```

**Operazioni di Lettura (Read)** Le query di selezione utilizzano il lock di lettura condiviso (`gen_rlock()`), consentendo letture simultanee da parte di più thread o coroutine senza blocchi reciproci.

```
def get_test_sequence_by_name(self, name):
    try:
        with self.locker.gen_rlock():
            self.conn.row_factory = sqlite3.Row
            cursor = self.conn.execute(
                "SELECT * FROM test_sequence WHERE name = ?",
                (name,)
            )
            row = cursor.fetchone()
            return {
                "success": True,
                "data": dict(row) if row else None
            }
except Exception as e:
```

```
    return {"success": False, "data": str(e)}
```

L'introduzione di un helper dinamico generico anche per le query di selezione (simile a quello adottato per la fase di *Update*) rappresenta un possibile sviluppo futuro per snellire ulteriormente la codebase.

## 3.5 Gestione del File System

La persistenza fisica dei file binari è affidata al modulo `FileManager`. Questo componente a basso livello si occupa di astrarre le operazioni di I/O sul disco.

### 3.5.1 Concorrenza di I/O sul File System

L'accesso concorrente e asincrono in lettura e scrittura al file system rappresenta uno degli aspetti più critici dell'architettura applicativa. Senza un idoneo meccanismo di sincronizzazione, le operazioni simultanee su uno stesso file rischiano di causare corruzione dei dati, corse di scrittura (*write races*) o eccezioni di blocco a livello di sistema operativo.

Per risolvere il problema garantendo al contempo un elevato grado di parallelismo, è stata adottata una strategia di *gestione dei lock basata su singola risorsa (per-path dynamic locking)*. Poiché è del tutto lecito eseguire operazioni di I/O simultanee su file distinti, l'obiettivo non è bloccare l'intero file system, ma applicare il pattern dei lettori/scrittori (*Readers-Writers*) unicamente al singolo file coinvolto nell'operazione.

**Struttura e Architettura del Modello** La gestione dinamica dei lock si basa sull'interazione tra la classe ausiliaria `ResourceLock` e una tabella di tracciamento centralizzata `self.resources` (dizionario Python):

```
class ResourceLock:
    """
    Rappresenta il lock associato a una specifica risorsa (file).
    Incapusla il meccanismo RWLock e un contatore di riferimenti
    per la gestione della memoria.
    """
    def __init__(self):
        self.lock = rwlock.RWLockFair()
        self.users = 0
```

Ogni risorsa viene identificata in modo univoco dal suo percorso assoluto nel file system (`os.path.abspath(path)`). Il ciclo di vita dei lock è regolato attraverso le due casistiche principali:

1. *Prima acquisizione o risorsa non tracciata*: se la risorsa richiesta non è presente nel dizionario `self.resources`, viene istanziata una nuova risorsa `ResourceLock`, inserita nel dizionario e associata al percorso.
2. *Acquisizione concorrente*: se la risorsa è già presente, viene riutilizzata l'istanza esistente di `ResourceLock`, consentendo la condivisione del medesimo lock tra i diversi thread o coroutine in attesa.

Per evitare perdite di memoria (*memory leak*), la classe mantiene un contatore di riferimenti (`resource.users`). Quando l'ultimo thread rilascia la risorsa e il valore del contatore scende a zero, la chiave viene rimossa dal dizionario `self.resources`.

**Implementazione dei Metodi di Acquisizione e Rilascio** L'accesso al dizionario condiviso dei lock è protetto a sua volta da un ulteriore lock globale interno (`self.resources_lock`), assicurando la thread-safety durante l'allocazione e la deallocazione dei singoli lock:

```
def _get_resource(self, path: str) -> tuple[str, ResourceLock]:
    """
    Recupera o crea il lock associato al path specificato.
    Incrementa il riferimento degli utilizzatori attivi.
    """
    path = os.path.abspath(path)

    with self.resources_lock:
        resource = self.resources.get(path)

        if resource is None:
            resource = ResourceLock()
            self.resources[path] = resource

        resource.users += 1

        return path, resource


def _release_resource(self, path: str, resource: ResourceLock):
    """
    Rilascia il riferimento al ResourceLock.
    Elimina la voce dal dizionario se il contatore utenti si azzera.
    """
    with self.resources_lock:
        current = self.resources.get(path)

        # Controllo di sicurezza: se la risorsa e stata riallocata
        # nel frattempo, non altera il dizionario.
        if current is not resource:
            return
```

```
        resource.users -= 1

        if resource.users == 0:
            del self.resources[path]
```

**Pattern di Utilizzo nell'Applicazione** Durante il normale flusso operativo, le funzioni del modulo accedono al file system integrando i due livelli di protezione (il gestore delle risorse e l'ambiente `rwlock`):

```
# 1. Recupero del lock dinamico specifico per il percorso
path, resource = self._get_resource(target_path)

try:
    # 2. Acquisizione del lock in scrittura (esclusivo) o in lettura (condiviso)
    with resource.lock.gen_wlock():
        # Esecuzione delle operazioni di I/O sul file
        with open(path, "w") as f:
            f.write(data)
finally:
    # 3. Rilascio garantito della risorsa e aggiornamento del Reference Counting
    self._release_resource(path, resource)
```

### 3.5.2 Specifica Dettagliata delle Funzioni

La classe `FileManager` incapsula le operazioni fondamentali sul file system seguendo il paradigma CRUD. Ciascun metodo integra il meccanismo di gestione dei lock dinamici (`_get_resource` e `_release_resource`) descritto in precedenza, garantendo l'accesso sicuro e atomico alle risorse in contesti di concorrenza.

**Operazione di Creazione (Create)** Il metodo `create_file` riceve il percorso relativo della risorsa e i dati in formato binario. Prima dell'I/O effettivo, la procedura garantisce l'esistenza della struttura di directory sottostante creando eventuali cartelle padre mancanti (`os.makedirs`), ed esegue la scrittura del file sotto protezione di un lock esclusivo in scrittura (`gen_wlock()`).

```
def create_file(self, path: str, data: bytes):
    path, resource = self._get_resource(path)

    try:
        with resource.lock.gen_wlock():
            # Crea le cartelle padre se non ancora esistenti
            directory = os.path.dirname(path)
            if directory:
                os.makedirs(directory, exist_ok=True)
```

```
            with open(path, "wb") as file:
                file.write(data)

    finally:
        self._release_resource(path, resource)
```

**Operazione di Aggiornamento (Update)** Il metodo `edit_file` si occupa di sovrascrivere il contenuto di un file preesistente. Acquisito il lock esclusivo di scrittura per il percorso specificato, il file viene aperto in modalità binaria con troncamento, garantendo l'aggiornamento atomico del payload.

```
def edit_file(self, path: str, data: bytes):
    path, resource = self._get_resource(path)

    try:
        with resource.lock.gen_wlock():
            with open(path, "wb") as file:
                file.write(data)

    finally:
        self._release_resource(path, resource)
```

**Operazione di Eliminazione (Delete)** La funzione `delete_file` gestisce la rimozione fisica del file. Previa verifica della reale esistenza della risorsa sul file system (`os.path.exists`), la cancellazione viene eseguita all'interno del blocco con lock di scrittura per prevenire corse critiche (*race conditions*) con altre operazioni concorrenti.

```
def delete_file(self, path: str):
    path, resource = self._get_resource(path)

    try:
        with resource.lock.gen_wlock():
            if os.path.exists(path):
                os.remove(path)

    finally:
        self._release_resource(path, resource)
```

**Operazione di Lettura e Download (Read)** La fruizione del file avviene tramite la funzione `download_file`, la quale combina il meccanismo di lettura condivisa (`gen_rlock()`) con lo streaming asincrono fornito da FastAPI (`StreamingResponse`). La lettura del file avviene a blocchi (*chunk*) da 1 MB per ottimizzare l'uso della memoria principale durante il trasferimento di artefatti di grandi dimensioni. Il rila-

scio del lock viene differito al completamento del generatore `stream()`, assicurando la coerenza del file per l'intera durata dello scaricamento.

```
def download_file(self, path: str, artifact_name: str):
    full_path = os.path.join(path, artifact_name)
    path, resource = self._get_resource(full_path)

    def stream():
        try:
            with resource.lock.gen_rlock():
                with open(path, "rb") as f:
                    while chunk := f.read(1024 * 1024):
                        yield chunk
        finally:
            self._release_resource(path, resource)

    return StreamingResponse(
        stream(),
        media_type="application/octet-stream",
        headers={
            "Content-Disposition": (
                f"attachment; filename*=UTF-8''{quote(artifact_name, safe='')}"
            ),
            "Content-Length": str(os.path.getsize(path)),
        },
    )
```

# Capitolo 4

# Front-end

Il Front-end ha la responsabilità di fornire un'interfaccia utente per la visualizzazione e la manipolazione dei dati esposti dalle API. Come mostrato nella Figura 2.4, il modulo adotta un'architettura basata sul pattern MVC. Nel corso di questo capitolo verranno analizzati nel dettaglio i componenti e le dinamiche di tale architettura.

## 4.1 Librerie e Dipendenze

Come anticipato nei capitoli precedenti, al fine di mantenere l'architettura del Front-end essenziale e leggera, si è scelto di sviluppare l'applicazione in Vanilla JavaScript, evitando l'introduzione di framework complessi. Ciononostante, per velocizzare lo sviluppo ed evitare la reinvenzione di funzionalità standard, sono state integrate alcune librerie di terze parti, riassunte nella Tabella 4.1.

Tabella 4.1: Librerie principali del Front-end e relative motivazioni d'uso.

| **Nome Libreria** | **Funzione** | **Motivazione dell'utilizzo** |
|---|---|---|
| `Bootstrap`[8] | Framework CSS/UI | Consente la realizzazione rapida di un'interfaccia utente *responsive*, uniforme e coerente con le linee guida moderne di usabilità. |
| `JSZip`[10] | Manipolazione archivi ZIP | Permette la decompressione e la lettura client-side degli archivi ZIP caricati dall'utente senza gravare sul Back-end. |
| `jQuery`[11] / `AJAX` | Manipolazione DOM | Semplifica la sintassi per la gestione degli eventi nel DOM. |

## 4.2 Model

Il livello di *Model* racchiude le strutture dati del sistema e gestisce interamente la logica di accesso, reperimento e persistenza delle informazioni sul client.

### 4.2.1 Modulo API Client

La directory `/static/api` contiene i moduli JavaScript incaricati di orchestrare e astrarre le interazioni di rete verso il Back-end. La gestione delle chiamate alle API RESTful è suddivisa nei seguenti file principali:

- `client.js`: Rappresenta il punto di accesso centralizzato per tutte le comunicazioni di rete. Implementa la funzione fondamentale `request()`, responsabile dell'esecuzione effettiva della chiamata HTTP tramite la Fetch API. La funzione accetta in ingresso l'endpoint target e un oggetto contenente le opzioni di configurazione della richiesta (es. metodo HTTP, parametri o *payload*). In output, restituisce uniformemente un oggetto strutturato con le chiavi:
  - `success` (*boolean*): indica l'esito della richiesta HTTP;
  - `data` (*Object*): contiene la risposta restituita dal server, includendo gli eventuali dettagli dei messaggi di errore.
- `TestApi.js` e `ArtifactApi.js`: Agiscono come moduli *wrapper* di livello superiore che espongono funzioni specifiche per la gestione delle entità del sistema (Test ed Artefatti). Ciascuna funzione accetta i parametri propri del dominio applicativo, li trasforma nella struttura appropriata e li inoltra alla funzione `request()` di `client.js`. In particolare, per le operazioni in lettura (`GET`) le opzioni della richiesta risultano vuote o limitate ai parametri di query, mentre per le operazioni di scrittura o caricamento (`POST`) viene costruito e incapsulato un oggetto di tipo `FormData`.

### 4.2.2 Gestione dello Stato con `store.js`

Il modulo `/state/store.js` ha la responsabilità di centralizzare e gestire lo stato globale dell'applicazione client, agendo come *Single Source of Truth* (SSOT). La scelta di adottare uno stato unico centralizzato è stata dettata dalla necessità di semplificare la propagazione dei dati tra i diversi moduli e componenti di *rendering*. In assenza di un contenitore di stato globale, il passaggio continuo di strutture dati complesse tramite argomenti di funzioni e valori di ritorno avrebbe introdotto un elevato grado di accoppiamento e complessità architetturale.

All'interno di `store.js`, lo stato è rappresentato da un oggetto globale denominato `store`, accessibile in sola lettura o modifica da tutte le componenti del Front-end. Poiché l'architettura dell'interfaccia utente è strutturata in modo che le operazioni di lettura e scrittura avvengano in modo sequenziale all'interno del

ciclo di eventi (*event loop*) del browser, non è stato necessario implementare meccanismi espliciti di controllo della concorrenza.

Di seguito vengono descritte nel dettaglio le principali strutture dati gestite all'interno dell'oggetto `store`:

**`artifacts` (*Array di Oggetti*)** Rappresenta la collezione di tutte le *repository* contenenti gli artefatti del sistema. Ogni elemento dell'array è descritto dalla seguente struttura:

- `id`: identificativo univoco della *repository*;
- `name`: denominazione della *repository*;
- `created_at`: data e ora di creazione della *repository*;
- `files`: array di oggetti rappresentanti i file contenuti nell'artefatto, dove ciascun elemento traccia le informazioni di dettaglio quali `id`, `creator` e `created_at`.

**`defaultRepoId` (*String / Integer*)** Mantiene l'identificativo univoco della *repository* predefinita selezionata durante le fasi di caricamento dei nuovi artefatti. Garantisce che i file inviati tramite l'interfaccia vengano associati correttamente alla destinazione prescritta.

**`selectedFiles` (*Array di Coppie di Stringhe*)** Gestisce la lista temporanea dei file selezionati dall'utente per il caricamento e la configurazione delle *Test Sequences*. Ogni elemento è rappresentato da una coppia di valori [Nome File, Tipo File], in cui la seconda stringa qualifica la natura del file caricato. Le tipologie di file supportate dal sistema comprendono:

- `zip`: archivi compressi contenenti dati o risorse esecutive;
- `excel`: fogli di calcolo per definizioni o dati di test;
- `Features_TestCases`: specifica dei casi di test e delle funzionalità;
- `Coverage`: report e metriche di copertura del codice;
- `Requirements_JSON`: definizioni strutturate dei requisiti in formato JSON;
- `TestSequenceNames`: elenchi descrittivi dei nomi delle sequenze di test.

**`featuresTestCasesLocal` (*Array di Oggetti*)** Contiene l'elenco temporaneo delle funzionalità e dei relativi casi di test estratti durante il parsing dei file. Ogni elemento dell'array possiede i seguenti campi:

- `id`: identificativo locale temporaneo dell'associazione;
- `feature`: nome o descrizione della funzionalità;

- `test_case`: identificativo o nome del caso di test associato.

**`coverageDataLocal` (*Array di Oggetti*)** Raccoglie i dati temporanei relativi alle metriche di copertura estratti dai report elaborati prima dell'invio definitivo al server. Viene popolato durante il parsing dei file di tipo `Coverage` e funge da struttura intermedia per la correlazione e la derivazione automatica delle entità. Ogni elemento dell'array è definito dalla seguente struttura:

- `ts_name`: denominazione o identificativo della sequenza di test (*Test Sequence*) a cui le metriche di copertura fanno riferimento;
- `feature`: identificativo della funzionalità di sistema analizzata;
- `test_case`: identificativo del caso di test associato alla coppia di copertura.

**`requirementsDataLocal` (*Array di Oggetti*)** Contiene la lista dei requisiti di sistema estratti dai file JSON. Ogni oggetto dell'array possiede i seguenti campi:

- `id`: identificativo locale del requisito;
- `name`: denominazione del requisito;
- `tags`: etichette e metadati associati al requisito;
- `test_cases`: array di oggetti rappresentanti i casi di test collegati al requisito, in cui ciascun elemento contiene l'identificativo del test case (`id`).

**`testSequenceNamesLocal` (*Array di Oggetti*)** Contiene la lista delle sequenze di test individuate nei file elaborati. Ogni oggetto dell'array possiede i seguenti campi:

- `id`: identificativo locale della sequenza di test;
- `ts_name`: nome della sequenza di test;
- `train_data_set`: dati di addestramento o di configurazione associati;
- `latest`: flag o indicatore relativo allo stato di aggiornamento della sequenza;
- `test_cases`: array di oggetti rappresentanti i casi di test collegati alla sequenza, in cui ciascun elemento contiene l'identificativo del test case (`id`).

**`versionDataLocal` (*Array di Oggetti*)** Memorizza le informazioni sulle versioni dei file da inviare al server. Ogni elemento dell'array possiede i seguenti campi:

- `current_file`: File con i dettagli del file corrente, a sua volta strutturato insieme a:
  - `hashToSend`: Hash univoco

  - `externalId`: identificativo della Test Sequence
  - `globalName`: nome del creatore
  - `description`: descrizione della versione
- `previous`: identificativo o riferimento alla versione del file precedente.

**`testSequenceRemote` (*Array di Oggetti*)** Rappresenta la collezione delle sequenze di test restituite dal backend. Ciascun elemento nell'array rispecchia la struttura dei dati memorizzata nel database:

- `id`: identificativo univoco della sequenza di test (*numero*);
- `name`: nome o denominazione della sequenza di test (*stringa*);
- `subsystem`: sotto-sistema di riferimento o modulo di appartenenza (*stringa*);
- `train_data_set`: configurazione del set dati di addestramento (*stringa*, accetta unicamente i valori: `'generic'`, `'gamma1'`, `'gamma2'`, `'lambda1'`, `'lambda2'`, `'lambda3'`);
- `latest`: hash della versione più recente associata, che fa riferimento al campo `hash` di `versionRemote` (*stringa*).

**`versionRemote` (*Array di Oggetti*)** Traccia la cronologia delle versioni dei file salvati sul server:

- `hash`: codice identificativo univoco della versione del file (*stringa*);
- `ts_id`: identificativo della sequenza di test associata, che fa riferimento al campo `id` di `testSequenceRemote` (*numero*);
- `filename`: nome del file memorizzato (*stringa*);
- `created_on`: data di creazione della versione (*stringa*);
- `created_by`: utente o sistema che ha registrato la versione (*stringa*);
- `notes`: note o dettagli descrittivi aggiuntivi (*stringa*);
- `previous_version`: hash della versione precedente nella catena di modifiche, che fa riferimento al campo `hash` di un altro oggetto in `versionRemote` (*stringa*).

**`featureTestCaseRemote` (*Array di Oggetti*)** Mantiene l'insieme delle associazioni univoche tra funzionalità e casi di test registrati:

- `id`: identificativo univoco dell'associazione (*numero*);
- `feature`: codice numerico o identificativo della funzionalità (*numero*);

- `test_case`: codice numerico o identificativo del caso di test (*numero*). La combinazione di `feature` e `test_case` è univoca.

**`requirementRemote` (*Array di Oggetti*)** Contiene l'elenco dei requisiti di sistema. Nel modello JavaScript del client, i tag associati dalla tabella correlata vengono aggregati direttamente all'interno dell'oggetto:

- `id`: identificativo univoco del requisito (*numero*);
- `name`: nome o descrizione sintetica del requisito (*stringa*);
- `tags`: lista delle etichette o tag associati al requisito (*array di stringhe*).

**`testSequenceFeatureTestCaseRemote` (*Array di Oggetti*)** Rappresenta la struttura di collegamento molti-a-molti tra le sequenze di test e i casi di test associati alle feature:

- `ts_id`: identificativo della sequenza di test, che fa riferimento al campo `id` di `testSequenceRemote` (*numero*);
- `ft_id`: identificativo dell'associazione feature-test case, che fa riferimento al campo `id` di `featureTestCaseRemote` (*numero*).

**`featureTestCaseRequirementRemote` (*Array di Oggetti*)** Rappresenta la struttura di collegamento molti-a-molti tra i casi di test associati alle feature e i requisiti soddisfatti:

- `ft_id`: identificativo dell'associazione feature-test case, che fa riferimento al campo `id` di `featureTestCaseRemote` (*numero*);
- `r_id`: identificativo del requisito, che fa riferimento al campo `id` di `requirementRemote` (*numero*).

**`errorLog` (*Array di Oggetti*)** Mantiene la cronologia degli errori di runtime intercettati dall'applicazione durante l'esecuzione del client. Ciascun oggetto nell'array rappresenta una singola segnalazione d'errore ed è strutturato come segue:

- `id`: codice numerico identificativo generato in modo casuale al momento della tracciamento dell'errore (*numero*);
- `time`: orario di registrazione dell'errore formattato secondo le impostazioni locali dell'utente (*stringa*);
- `message`: messaggio descrittivo o sintesi della tipologia di errore riscontrata (*stringa*);
- `stacktrace`: traccia dettagliata dello stack di esecuzione associata all'errore; qualora non fosse disponibile o specificata, assume il valore predefinito `"No stacktrace available"` (*stringa*).

**`hashList` (*Set di Stringhe*)** Raccoglie in un insieme (*Set*) privo di duplicati l'elenco di tutte le impronte digitali (hash MD5) individuate o elaborate dall'applicazione

**`globalName` (*Stringa*)** Memorizza il nome dell'operatore che sta effettuando la sessione corrente di caricamento dei nuovi file.

**`idFeaturesData` (*Array di Numeri / Stringhe*)** Collezione indicizzata contenente unicamente gli identificativi univoci delle (*feature test cases*). La struttura viene popolata ed elaborata per ottimizzare le prestazioni del client, evitando la necessita di scansionare ripetutamente le strutture dati più complesse (`featuresTestCasesLocal`) durante l'esecuzione di operazioni frequenti di autocompletamento nell'interfaccia.

**`localToReal` (*Oggetto*)** Rappresenta la mappa di convergenza tra gli identificativi temporanei generati dall'interfaccia client e gli identificativi effettivi assegnati dal database remoto a seguito della persistenza dei dati. È suddiviso in dizionari specifici per ciascun ambito di entità:

- `ts`: dizionario di corrispondenza per le Test Sequence, contenente le coppie valore (*chiave-valore*, `ID_locale` → `ID_remoto`);
- `ftc`: dizionario di corrispondenza per le associazioni feature–test case, contenente le coppie valore (*chiave-valore*, `ID_locale` → `ID_remoto`);
- `req`: dizionario di corrispondenza per i Requirements, contenente le coppie valore (*chiave-valore*, `ID_locale` → `ID_remoto`).

**`errorCheck` (*Booleano*)** Flag di configurazione che determina l'attivazione della verifica preventiva degli errori e delle validazioni di coerenza sui dati prima delle operazioni di invio dei dati. Di default assume valore `true`:

- `true`: abilita l'esecuzione completa di tutti i controlli di validazione e integrità logica sui dati;
- `false`: disabilita la maggior parte dei controlli preventivi, questo per testare le API anche in casi non controllati.

## 4.3 View

Il componente di *View* all'interno dell'architettura client ha il compito fondamentale di generare dinamicamente e visualizzare il codice HTML dell'interfaccia utente. Data l'elevata complessità strutturale e la differente natura operativa legate alla gestione degli *Artifact* e delle *Test Sequence*, i moduli dedicati al rispettivo *rendering* sono stati completamente separati.

Tuttavia, nonostante questa netta suddivisione funzionale, la logica architetturale di fondo e le modalità di interazione con lo stato globale dell'applicazione rimangono omogenee e coerenti tra tutti i componenti. Nelle sottosezioni seguenti verranno analizzate nel dettaglio le due macro-tipologie di *View* sviluppate per il progetto.

### 4.3.1 Dashboard

Le *Dashboard* sono collocate all'interno della directory `/templates` e rappresentano i punti di ingresso dell'interfaccia utente, costituendo il primo blocco di codice HTML caricato al momento dell'accesso all'applicazione. Le Dashboards si differenziano dai Renderers siccome sono componenti statici. Il sistema prevede due *Dashboard* distinte in base all'ambito operativo:

- `dashboardTests.html`: dedicata alla gestione, visualizzazione e tracciamento delle *Test Sequence*;
- `dashboardArtifacts.html`: incentrata sulla consultazione e gestione degli *Artifact*.

Nonostante la separazione funzionale dei domini applicativi, entrambe le pagine condividono una struttura comune per la gestione dei feedback visuali e dello stato di avanzamento delle operazioni. In particolare, integrano i seguenti elementi dell'interfaccia:

**Overlay** Per garantire un'esperienza utente reattiva e trasparente durante l'esecuzione di operazioni asincrone, l'interfaccia integra componenti dedicati alla notifica dello stato e al tracciamento degli eventi lato client. La Tabella 4.2 descrive nel dettaglio la funzione di ciascun elemento.

Tabella 4.2: Componenti di interfaccia per il feedback visivo e la gestione degli errori.

| **Componente** | **Tipologia** | **Descrizione e Funzione** |
|---|---|---|
| **Overlay di Stato** | Modal a comparsa | Fornisce un riscontro visivo immediato sull'esito di operazioni brevi (es. conferme di salvataggio o errori) tramite icone animate. |
| **Overlay di Avanzamento** | Modal con *Progress Bar* | Monitora i processi asincroni a lungo termine (es. parsing di file complessi), mostrando percentuale e stima dei tempi di completamento. |
| **Registro Errori** | Contenitore dinamico (*Error Log*) | Elemento inizialmente vuoto, popolato in tempo reale con le eccezioni di runtime intercettate dal client (`errorLog`). |

**Differenze Funzionali tra le Dashboard** In aggiunta ai componenti comuni di feedback e tracciamento, le due *Dashboard* si differenziano per la gestione specifica del flusso operativo e della presentazione delle risorse, come sintetizzato nella Tabella 4.3.

Tabella 4.3: Differenze funzionali e strutturali tra le due Dashboard.

| **Elemento** | **Descrizione e Comportamento** |
|---|---|
| **Modal di caricamento dedicati** | Ciascuna pagina implementa una finestra di dialogo modale specifica per l'upload dei dati, adattando campi di input e validazioni al dominio di riferimento. |
| **Container di visualizzazione indipendenti** | Elementi contenitori dedicati la cui struttura DOM viene popolata e aggiornata dinamicamente in base alla risorsa restituita dalle API. |

**Renderers** I componenti dedicati alla manipolazione dinamica dell'interfaccia si trovano all'interno della directory `static/components`. A differenza degli elementi statici dell'interfaccia, i *Renderers* sono moduli dinamici il cui output varia in base ai dati di input ricevuti in ingresso; essi interagiscono direttamente con il DOM, alterando le strutture specifiche delle due *Dashboard*.

La Tabella 4.4 ne sintetizza l'architettura e la relativa responsabilità funzionale.

Tabella 4.4: Architettura e responsabilità dei componenti *Renderer*.

| **Nome Componente** | **Descrizione e Funzione** |
|---|---|
| `logger.js` | Fornisce le funzioni per la formattazione e la visualizzazione in tempo reale dei log applicativi e di runtime. |
| `rendererFileSystem.js` | Espone le funzioni dedicate al rendering degli *Artifacts*, rappresentandone la struttura gerarchica con uno stile grafico a file system. |
| `rendererProgressBar.js` | Incapsula una classe per il controllo e la gestione dinamica dello stato di avanzamento della *Progress Bar*. |
| `rendererAdders/` | Directory contenente i *renderers* preposti alla costruzione dell'interfaccia per il caricamento e l'aggiunta delle *Test Sequence*. |
| `rendererViewers/` | Directory contenente i *renderers* preposti alla visualizzazione e all'ispezione dei dati remoti relativi alle *Test Sequence*. |

Di seguito vengono descritti nel dettaglio i singoli moduli contenuti all'interno delle directory `rendererAdders/` e `rendererViewers/`.

**Moduli di caricamento (`rendererAdders/`)** La Tabella 4.5 sintetizza i componenti responsabili della costruzione dell'interfaccia per la gestione del caricamento dei dati riguardante le *Test Sequence Repository*.

Tabella 4.5: Componenti *Renderer* per l'aggiunta delle risorse (`rendererAdders/`).

| **Modulo** | **Descrizione e Funzione** |
|---|---|
| `featureRenderer.js` | Gestisce il rendering e la visualizzazione delle *Feature* caricate dall'utente. |
| `fileRenderer.js` | Gestisce la visualizzazione dell'elenco dei file allegati. |
| `requirementRenderer.js` | Mostra e gestisce la visualizzazione dei *Requirement* inseriti. |
| `sequenceRenderer.js` | Rendering dei componenti per l'aggiunta delle *Test Sequence*. |
| `versionRenderer.js` | Visualizza e organizza le informazioni relative alle versioni selezionate. |

**Moduli di visualizzazione remota (`rendererViewers/`)** La Tabella 4.6 descrive i componenti dedicati alla consultazione e la modifica dei dati remoti restituiti dal server.

Tabella 4.6: Componenti *Renderer* per la visualizzazione dei dati remoti (`rendererViewers/`).

| **Modulo** | **Descrizione e Funzione** |
|---|---|
| `configRenderer.js` | Rendering dell'interfaccia di configurazione generale della *Dashboard*. |
| `featureTestCaseRenderer.js` | Rendering della tabella contenente le *Feature Test Case* registrate sul server. |
| `ftRequirementLinkRenderer.js` | Mostra la struttura di associazione tra le *Feature Test Case* e i relativi *Requirement*. |
| `requirementRenderer.js` | Rendering della tabella dei *Requirement* memorizzati sul server. |
| `testSequenceRenderer.js` | Rendering della tabella riepilogativa delle *Test Sequence* del server. |
| `tsFtLinkRenderer.js` | Mostra la tabella di collegamento tra le *Test Sequence* e le rispettive *Feature Test Case*. |
| `versionRenderer.js` | Gestisce il rendering della tabella per il tracciamento e la consultazione delle versioni. |

## 4.4 Controller

Il livello dei *Controller* ha la responsabilità di intercettare le interazioni e le richieste di input dell'utente, elaborararle invocando le opportune logiche di gestione, aggiornare lo stato del *Model* e coordinare l'esecuzione flessibile dell'interfaccia client.

### 4.4.1 Main Controller

La logica di coordinamento principale del lato client è implementata nei due file `static/mainArtifacts.js` e `static/mainTests.js`, preposti rispettivamente alla gestione del repository degli *Artifact* e delle *Test Sequence*.

L'architettura interna di entrambi i moduli è organizzata in tre sezioni funzionali distinte:

**Esposizione di variabili globali** In questa sezione vengono dichiarate e definite le variabili di stato e le istanze condivise, rendendo i dati e le utility acces-

sibili globalmente all'interno dello script per supportare l'esecuzione delle diverse funzioni.

**Preparazione e logica di rendering** Viene definita la funzione principale di aggiornamento dell'interfaccia (`renderData()`). Tale funzione, ogni volta che viene invocata, si occupa di sincronizzare il DOM ri-renderizzando l'insieme dei dati aggiornati all'interno della pagina.

**Inizializzazione del sistema** Costituisce la fase di avvio del client. Vengono invocate le procedure per il caricamento delle configurazione dell'ambiente, inclusa la registrazione degli *Handler* di evento e il recupero iniziale dei dati dal server, culminando nell'invocazione della funzione `renderData()` per la prima presentazione visiva all'utente.

### 4.4.2 Handlers

Gli *Handler* costituiscono il livello di gestione degli eventi e dell'interazione utente: intercettano le azioni dell'interfaccia (es. click, digitazione, invio di moduli), elaborano la logica di controllo client-side e aggiornano lo stato interno del *Model*.

I moduli principali del livello sono collocati nella directory `/static/handlers`.

**Note di Progettazione Architetturale** Per la gestione dei dati specifici del repository delle *Test Sequence*, alcune funzioni di risposta agli eventi sono state collocate direttamente all'interno dei file *Renderer* corrispondenti (in particolare nella sezione finale dei relativi script). Sebbene questa scelta si discosti dal modello MVC puro (separazione netta tra *Controller* e *View*), è stata adottata per migliorare la leggibilità e la manutenibilità del codice, evitando un'eccessiva frammentazione dei file per funzioni ad accoppiamento stretto il cui unico compito è la sincronizzazione diretta tra dati remoti e variabili locali dell'interfaccia.

**Struttura dei File Handler** La Tabella 4.7 sintetizza i moduli collocati all'interno della cartella `/static/handlers` e le rispettive responsabilità funzionali.

Tabella 4.7: Panoramica dei moduli *Handler* e relative responsabilità.

| **Modulo** | **Descrizione e Funzione** |
|---|---|
| `AutoCompleteHelper.js` | Implementa le funzionalità di autocompletamento dinamico durante l'input di testo nelle caselle di inserimento. |
| `buttonHandlersArtifact.js` | Configura e registra gli *event listener* per tutti i pulsanti dell'interfaccia dell'Artifact Repository. |
| `buttonHandlersTest.js` | Registra gli *handler* dedicati ai pulsanti per l'upload e la gestione dei dati nella Test Sequence Repository. |
| `configHandler.js` | Gestisce la configurazione dell'applicazione e la commutazione dinamica tra le schermate di visualizzazione dei dati remoti. |
| `successHandler.js` | Coordina il comportamento dell'overlay di sistema per la visualizzazione dei messaggi di successo o di errore. |
| `uploadExecutorHandler.js` | Gestisce la preparazione del payload e l'invio asincrono delle richieste di caricamento verso il server. |
| `uploadPlannerHandler.js` | Presiede alla lettura e al parsing dei file selezionati in locale, convertendoli nelle strutture dati utilizzate dal client. |

**Moduli di Utilità (`Utils`)** L'ultima componente condivisa dell'architettura client-side è rappresentata dalla directory `/static/utils`. Questa cartella raccoglie una serie di moduli helper e funzioni di servizio trasversali, riutilizzate nelle diverse sezioni dell'applicazione per supportare la manipolazione dei dati, le operazioni di I/O e la gestione dell'interfaccia.

La Tabella 4.8 illustra i moduli presenti nella cartella `Utils` e le rispettive funzionalità.

Tabella 4.8: Panoramica dei moduli di utilità (`Utils`) e relative funzionalità.

| **Modulo** | **Descrizione e Funzione** |
|---|---|
| `cookieHelper.js` | Espone le funzioni per la lettura e la scrittura dei cookie, impiegati per la persistenza locale delle preferenze di configurazione dell'utente. |
| `csvParser.js` | Fornisce le routine per il parsing e la validazione dei file in formato CSV durante la fase di caricamento locale dei dati. |
| `dataConnector.js` | Contiene le funzioni di utilità per la strutturazione e il collegamento dei dati in pseudo-tabelle relazionali, richiamate durante l'importazione sia locale sia remota. |
| `domUtils.js` | Raccoglie funzioni di supporto per la manipolazione del DOM, inclusa la gestione dell'overlay visivo durante le operazioni di caricamento. |
| `formatter.js` | Mette a disposizione funzioni per la conversione dei tipi di dato e la formattazione avanzata delle stringhe per l'interfaccia. |
| `hash.js` | Implementa le funzioni di hashing MD5, impiegate per la generazione univoca dei nomi e degli identificativi dei file di versione. |
| `mathUtils.js` | Contiene utility matematiche di supporto, tra cui funzioni per la verifica dell'appartenenza di valori numerici a specifici intervalli (*range*). |
| `zipScanner.js` | Fornisce le funzionalità per la scansione, la decompressione e la lettura del contenuto dei file in formato compressed archive (`.zip`). |

# Capitolo 5

# Conclusioni

Il progetto si è concluso con successo, soddisfacendo i requisiti operativi e funzionali concordati. Prima della definitiva integrazione con i sistemi del partner industriale, tuttavia, sarà necessario un deployment "pilota" in un ambiente di sviluppo ridotto per far emergere criticità legate ai flussi di lavoro reali.

Dal punto di vista formativo, l'esperienza ha rappresentato un'importante occasione di crescita professionale e tecnica. Lo sviluppo completo dell'architettura, dal design del database relazionale e del Back-end asincrono in Python/FastAPI fino alla realizzazione dell'interfaccia utente dinamica lato Front-end, ha consentito di consolidare competenze pratiche nell'ingegneria del software, nella gestione della concorrenza e nell'organizzazione di sistemi complessi.

## 5.1 Sviluppi Futuri

A causa dei vincoli temporali tipici dell'attività, il software sviluppato presenta alcune disomogeneità implementative che costituiscono il naturale punto di partenza per futuri interventi di refactoring e ottimizzazione. Ciononostante, l'architettura realizzata rappresenta una solida base per l'evoluzione verso un'infrastruttura di grado aziendale.

I principali sviluppi futuri e i possibili ampliamenti funzionali includono:

- **Integrazione con l'infrastruttura esistente:** Avendo condotto le fasi di test in un ambiente isolato, uno dei primi passi operativi consisterà nel collaudo sul campo e nell'integrazione completa della piattaforma con i sistemi e i flussi di lavoro preesistenti;
- **Automazione della pipeline di aggiornamento:** Le *Test Sequence* ufficiali vengono attualmente prelevate dal portale dell'ERA e convertite nel formato supportato dal simulatore. Risulta coerente automatizzare l'intero flusso di download, parsing e conversione, consentendo il caricamento schedulato dei file aggiornati ed eliminando la necessità di interventi manuali;

- **Autenticazione:** L'assenza di un modulo di autenticazione strutturato rende attualmente complessa la tracciabilità e la gestione della paternità (*ownership*) delle sequenze di test e degli artefatti. Si prevede l'introduzione di un modello di controllo degli accessi basato sui ruoli (RBAC) e la generazione di *API key* dedicate per le pipeline di *Continuous Integration* (CI/CD), garantendo la sicurezza delle operazioni sui repository;
- **Ottimizzazione e refactoring del codice:** La base di codice presenta alcune aree di miglioramento legate alla ridondanza della logica di interrogazione (come nel caso del modulo `SQLmanager.py`) e alla gestione di grandi volumi di dati (es. il componente di autocompletamento in fase di upload). Tali moduli richiederanno interventi mirati di ingegnerizzazione per garantire elevata scalabilità e manutenibilità nel lungo periodo.

# Appendice A

# API Artifact Repository (/ar/)

La sezione **Artifact Repository** raccoglie le interfacce per la gestione, l'archiviazione e la consultazione degli artifact e dei relativi file memorizzati nel sistema.

## A.1 Dashboard

| Metodo | `EndPoint` | Risposta |
|---|---|---|
| GET | `/ar/dashboard` | Renderizzazione dell'interfaccia grafica principale (HTML) |

Tabella A.1: Dettagli chiamata API `/ar/dashboard`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## A.2 Aggiunta Artifact / File

| Metodo | `EndPoint` | Risposta |
|---|---|---|
| POST | `/ar/add` | Esito dell'operazione (JSON) |

Tabella A.2: Dettagli chiamata API `/ar/add`.

**Descrizione Parametri:**

- `artifact_repo_id` (*int, opzionale*): ID del repository di destinazione. Se non specificato, viene creato un nuovo repository.
- `artifact_file` (*UploadFile, opzionale*): Contenuto binario del file da caricare. Se omesso, viene creato unicamente il record del repository.
- `creator` (*string, opzionale*): Identificativo o nome dell'utente che effettua il caricamento.

## A.3 Rinomina Repository

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ar/rename_repository` | Esito dell'operazione (JSON) |

Tabella A.3: Dettagli chiamata API `/ar/rename_repository`.

**Descrizione Parametri:**

- `artifact_repo_id` (*int, obbligatorio*): ID del repository da aggiornare.
- `artifact_name` (*string, obbligatorio*): Nuova denominazione da assegnare al repository.

## A.4 Ricerca e Recupero File Artifact

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ar/get` | Oggetto o elenco strutturato di file Artifact (JSON) |

Tabella A.4: Dettagli chiamata API `/ar/get`.

**Descrizione Parametri:**

- `file_id` (*int, opzionale*): ID del singolo file da recuperare.
- `start_id` / `end_id` (*int, opzionali*): Intervallo di ID per la ricerca filtrata dei file.
- `artifact_id` (*int, opzionale*): ID dell'artifact di riferimento (utilizzato in combinazione con i nomi dei file).
- `file_name` (*string, opzionale*): Nome del singolo file da recuperare nell'artifact.
- `file_names` (*list[string], opzionale*): Elenco di nomi file da recuperare all'interno dell'artifact.

## A.5 Lista Tutti gli Artifact

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ar/list_artifacts` | Elenco contenente gli ID di tutti gli artifact memorizzati (JSON) |

Tabella A.5: Dettagli chiamata API `/ar/list_artifacts`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## A.6 Data di Creazione Artifact

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ar/get_creation_-date_artifact` | Data e ora di creazione dell'artifact (JSON) |

Tabella A.6: Dettagli chiamata API `/ar/get_creation_date_artifact`.

**Descrizione Parametri:**

- `artifact_id` (*int, obbligatorio*): ID dell'artifact di cui recuperare la data di creazione.

## A.7 Creatore Artifact

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ar/get_created_by_-artifact` | Utente autore dell'artifact (JSON) |

Tabella A.7: Dettagli chiamata API `/ar/get_created_by_artifact`.

**Descrizione Parametri:**

- `artifact_id` (*int, obbligatorio*): ID dell'artifact di cui recuperare l'autore.

## A.8 Elenco File in Artifact

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ar/list_files_in_-artifact` | Elenco dei file contenuti nell'artifact specificato (JSON) |

Tabella A.8: Dettagli chiamata API `/ar/list_files_in_artifact`.

**Descrizione Parametri:**

- `artifact_id` (*int, obbligatorio*): ID dell'artifact di cui elencare i file contenuti.

## A.9 Download Repository

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ar/download_repo` | Flusso di dati binario del repository compresso (ZIP) |

Tabella A.9: Dettagli chiamata API `/ar/download_repo`.

**Descrizione Parametri:**

- `repoId` (*string, obbligatorio*): ID del repository da scaricare.

## A.10 Download Singolo File

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ar/download_file` | Flusso di dati binari del file richiesto |

Tabella A.10: Dettagli chiamata API `/ar/download_file`.

**Descrizione Parametri:**

- `repoId` (*string, obbligatorio*): ID del repository contenitore.
- `fileId` (*int, obbligatorio*): ID del file specifico da scaricare.

## A.11 Download Artifact per Nome

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ar/download` | Flusso di dati dell'artifact indicato |

Tabella A.11: Dettagli chiamata API `/ar/download`.

**Descrizione Parametri:**

- `artifact_repo_id` (*string, obbligatorio*): ID del repository contenitore.
- `artifact_name` (*string, obbligatorio*): Nome dell'artifact da scaricare.

## A.12 Rimozione File / Artifact

| Metodo | EndPoint | Risposta |
|---|---|---|
| DELETE | `/ar/remove` | Esito dell'operazione di eliminazione (JSON) |

Tabella A.12: Dettagli chiamata API `/ar/remove`.

**Descrizione Parametri:**

- `artifact_id` (*int, opzionale*): ID dell'artifact completo da eliminare.
- `file_id` (*int, opzionale*): ID del singolo file da rimuovere (utilizzato qualora `artifact_id` non sia specificato).

# Appendice B

# Test Sequence Repository (/ts/)

La sezione **Test Sequence Repository** raccoglie le interfacce per la gestione, la consultazione e il tracciamento delle sequenze di test, delle versioni, dei requisiti e della coverage del sistema.

## B.1 Dashboard Test

| **Metodo** | **`EndPoint`** | **Risposta** |
|---|---|---|
| GET | `/ts/dashboard` | Renderizzazione interfaccia grafica principale (HTML) |

Tabella B.1: Dettagli chiamata API `/ts/dashboard`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## B.2 Aggiunta Test Sequence

| **Metodo** | **`EndPoint`** | **Risposta** |
|---|---|---|
| POST | `/ts/add_ts` | Esito dell'operazione (JSON) |

Tabella B.2: Dettagli chiamata API `/ts/add_ts`.

**Descrizione Parametri:**

- `name` (*string, obbligatorio*): Nome della sequenza di test.
- `training_data_set` (*string, obbligatorio*): Dataset di addestramento o riferimento utilizzato.
- `latest` (*string, opzionale*): Identificatore dell'ultima versione di riferimento.

## B.3 Eliminazione Test Sequence

| Metodo | EndPoint | Risposta |
|---|---|---|
| DELETE | `/ts/delete_ts` | Esito dell'operazione (JSON) |

Tabella B.3: Dettagli chiamata API `/ts/delete_ts`.

**Descrizione Parametri:**

- `ts_id` (*int, obbligatorio*): ID della sequenza di test da eliminare.

## B.4 Eliminazione Associazione Test Sequence e Feature Test

| Metodo | EndPoint | Risposta |
|---|---|---|
| DELETE | `/ts/delete_TsFt` | Esito dell'operazione (JSON) |

Tabella B.4: Dettagli chiamata API `/ts/delete_TsFt`.

**Descrizione Parametri:**

- `ts_id` (*int, obbligatorio*): ID della sequenza di test.
- `ft_id` (*int, obbligatorio*): ID del Feature Test Case associato.

## B.5 Aggiunta Feature Test Case

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ts/add_ftc` | Esito dell'operazione (JSON) |

Tabella B.5: Dettagli chiamata API `/ts/add_ftc`.

**Descrizione Parametri:**

- `feature` (*string, obbligatorio*): Denominazione della feature.
- `test_case` (*string, obbligatorio*): Descrizione o identificatore del caso di test.

## B.6 Associazione Requisito e Feature Case

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ts/add_requirement_-featureCase` | Esito dell'operazione (JSON) |

Tabella B.6: Dettagli chiamata API `/ts/add_requirement_featureCase`.

**Descrizione Parametri:**

- `r_id` (*int, obbligatorio*): ID del requisito.
- `ft_id` (*int, obbligatorio*): ID del Feature Test Case.

## B.7 Eliminazione Associazione Requisito e Feature Case

| Metodo | EndPoint | Risposta |
|---|---|---|
| DELETE | `/ts/delete_ft_requirement` | Esito dell'operazione (JSON) |

Tabella B.7: Dettagli chiamata API `/ts/delete_ft_requirement`.

**Descrizione Parametri:**

- `r_id` (*int, obbligatorio*): ID del requisito.
- `ft_id` (*int, obbligatorio*): ID del Feature Test Case.

## B.8 Associazione Test Sequence e Feature Case

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ts/add_testSequence_featureCase` | Esito dell'operazione (JSON) |

Tabella B.8: Dettagli chiamata API `/ts/add_testSequence_featureCase`.

**Descrizione Parametri:**

- `ts_id` (*int, obbligatorio*): ID della sequenza di test.
- `ft_id` (*int, obbligatorio*): ID del Feature Test Case.

## B.9 Aggiunta Requisito

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ts/add_reqs` | Esito dell'operazione (JSON) |

Tabella B.9: Dettagli chiamata API `/ts/add_reqs`.

**Descrizione Parametri:**

- `name` (*string, obbligatorio*): Nome del requisito.
- `tags` (*string, obbligatorio*): Tag di categorizzazione associati al requisito.

## B.10 Aggiunta Versione

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ts/add_version` | Esito dell'operazione (JSON) |

Tabella B.10: Dettagli chiamata API `/ts/add_version`.

**Descrizione Parametri:**

- `hashId` (*string, obbligatorio*): Hash identificativo univoco della versione.
- `ts_id` (*int, obbligatorio*): ID della sequenza di test associata.
- `created_by` (*string, obbligatorio*): Utente creatore della versione.
- `notes` (*string, opzionale*): Note descrittive della versione.
- `previous_version` (*string, opzionale*): Hash della versione precedente.
- `file` (*UploadFile, obbligatorio*): File binario relativo alla versione.

## B.11 Modifica File Versione

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ts/edit_file` | Esito dell'operazione (JSON) |

Tabella B.11: Dettagli chiamata API `/ts/edit_file`.

**Descrizione Parametri:**

- `hash` (*string, obbligatorio*): Hash della versione da aggiornare.
- `file` (*UploadFile, obbligatorio*): Nuovo file da caricare.

## B.12 Elenco ID Test Sequence

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_test_-sequence_ids` | Elenco degli ID delle sequenze di test (JSON) |

Tabella B.12: Dettagli chiamata API `/ts/get_test_sequence_ids`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## B.13 Recupero Test Sequence per ID

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_test_-sequence_by_id` | Dettagli della sequenza di test (JSON) |

Tabella B.13: Dettagli chiamata API `/ts/get_test_sequence_by_id`.

**Descrizione Parametri:**

- `ts_id` (*int, obbligatorio*): ID della sequenza di test da recuperare.

## B.14 Recupero Test Sequence per Intervallo di ID

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_test_sequence_by_id_range` | Elenco delle sequenze di test nell'intervallo (JSON) |

Tabella B.14: Dettagli chiamata API `/ts/get_test_sequence_by_id_range`.

**Descrizione Parametri:**

- `id_start` (*int, obbligatorio*): ID iniziale dell'intervallo.
- `id_end` (*int, obbligatorio*): ID finale dell'intervallo.

## B.15 Recupero Test Sequence per Nome

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_test_sequence_by_name` | Sequenza di test corrispondente (JSON) |

Tabella B.15: Dettagli chiamata API `/ts/get_test_sequence_by_name`.

**Descrizione Parametri:**

- `name` (*string, obbligatorio*): Nome della sequenza di test da cercare.

## B.16 Elenco Tutti i Test Sequence

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_all_test_sequence` | Elenco completo delle sequenze di test (JSON) |

Tabella B.16: Dettagli chiamata API `/ts/get_all_test_sequence`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## B.17 Elenco Hash delle Versioni

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_version_hashes` | Elenco degli hash di tutte le versioni (JSON) |

Tabella B.17: Dettagli chiamata API `/ts/get_version_hashes`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## B.18 Recupero Versione per Hash

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_version_by_hash` | Dettagli della versione (JSON) |

Tabella B.18: Dettagli chiamata API `/ts/get_version_by_hash`.

**Descrizione Parametri:**

- `hash` (*string, obbligatorio*): Hash della versione da recuperare.

## B.19 Download File per Hash

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_file_by_hash` | Flusso di dati del file |

Tabella B.19: Dettagli chiamata API `/ts/get_file_by_hash`.

**Descrizione Parametri:**

- `hash` (*string, obbligatorio*): Hash del file da scaricare.
- `zip` (*bool, opzionale*): Se true, restituisce il file compresso in archivio ZIP (Default: false).

## B.20 Download File per Hash Multipli

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_file_by_hashes` | Flusso di dati dei file |

Tabella B.20: Dettagli chiamata API `/ts/get_file_by_hashes`.

**Descrizione Parametri:**

- `hashes` (*string, obbligatorio*): Elenco o stringa di hash separati da virgola per il recupero multiplo.

## B.21 Recupero File per ID Test Sequence

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_files_by_ts_id` | Elenco file o flusso di dati ZIP |

Tabella B.21: Dettagli chiamata API `/ts/get_files_by_ts_id`.

**Descrizione Parametri:**

- `ts_id` (*string, obbligatorio*): ID della sequenza di test.

- `file_types` (*string, obbligatorio*): Filtro sui tipi di file richiesti.
- `zip` (*bool, obbligatorio*): Se true, impacchetta i file in un file ZIP.

## B.22 Recupero File per Intervallo di ID Test Sequence

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_files_by_ts_id_range` | Elenco file o flusso di dati ZIP |

Tabella B.22: Dettagli chiamata API `/ts/get_files_by_ts_id_range`.

**Descrizione Parametri:**

- `ts_id_start` (*string, obbligatorio*): ID iniziale della sequenza di test.
- `ts_id_end` (*string, obbligatorio*): ID finale della sequenza di test.
- `file_types` (*string, obbligatorio*): Filtro sui tipi di file richiesti.
- `zip` (*bool, obbligatorio*): Se true, impacchetta i file in un archivio ZIP.

## B.23 Recupero di Tutti i File

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_all_files` | Elenco file o flusso di dati ZIP |

Tabella B.23: Dettagli chiamata API `/ts/get_all_files`.

**Descrizione Parametri:**

- `file_types` (*string, obbligatorio*): Filtro sui tipi di file richiesti.
- `zip` (*bool, obbligatorio*): Se true, impacchetta i file in un archivio ZIP.

## B.24 Elenco Tutte le Versioni

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_all_versions` | Elenco completo delle versioni memorizzate (JSON) |

Tabella B.24: Dettagli chiamata API `/ts/get_all_versions`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## B.25 Elenco ID dei Feature Test Case

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_all_feature_-test_case_ids` | Elenco degli ID di tutti i Feature Test Case (JSON) |

Tabella B.25: Dettagli chiamata API `/ts/get_all_feature_test_case_ids`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## B.26 Elenco Tutti i Feature Test Case

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_all_feature_-test_case` | Elenco completo dei Feature Test Case (JSON) |

Tabella B.26: Dettagli chiamata API `/ts/get_all_feature_test_case`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## B.27 Recupero Feature Test Case per ID

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_feature_-test_case_by_id` | Dettagli del Feature Test Case (JSON) |

Tabella B.27: Dettagli chiamata API `/ts/get_feature_test_case_by_id`.

**Descrizione Parametri:**

- `ft_id` (*int, obbligatorio*): ID del Feature Test Case.

## B.28 Recupero ID Feature Test Case da Feature e Test Case

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_id_feature_-test_case_by_-feature_and_test_case` | ID del Feature Test Case (JSON) |

Tabella B.28: Dettagli chiamata API `/ts/get_id_feature_test_case_by_feature_-and_test_case`.

**Descrizione Parametri:**

- `feature` (*int, obbligatorio*): Identificatore numerico della feature.
- `test_case` (*int, obbligatorio*): Identificatore numerico del caso di test.

## B.29 Elenco ID dei Requisiti

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_all_-requirement_ids` | Elenco degli ID di tutti i requisiti (JSON) |

Tabella B.29: Dettagli chiamata API `/ts/get_all_requirement_ids`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## B.30 Recupero Requisito per ID

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_requirement_-by_id` | Dettagli del requisito (JSON) |

Tabella B.30: Dettagli chiamata API `/ts/get_requirement_by_id`.

**Descrizione Parametri:**

- `r_id` (*int, obbligatorio*): ID del requisito.

## B.31 Elenco Tutti i Requisiti

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_all_-requirements` | Elenco completo dei requisiti (JSON) |

Tabella B.31: Dettagli chiamata API `/ts/get_all_requirements`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## B.32 Elenco Tutte le Associazioni Test Sequence e Feature Case

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_all_test_-sequence_feature_-test_case` | Mappatura completa TS-FT (JSON) |

Tabella B.32: Dettagli chiamata API `/ts/get_all_test_sequence_feature_test_-case`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## B.33 Recupero Associazione Test Sequence e Feature Case

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_test_-sequence_feature_-test_case` | Dettagli dell'associazione specifica (JSON) |

Tabella B.33: Dettagli chiamata API `/ts/get_test_sequence_feature_test_case`.

**Descrizione Parametri:**

- `ts_id` (*int, obbligatorio*): ID della sequenza di test.
- `ft_id` (*int, obbligatorio*): ID del Feature Test Case.

## B.34 Recupero Associazioni Feature Case per ID Test Sequence

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_test_-sequence_feature_-test_case_by_ts_id` | Elenco delle associazioni per il TS indicato (JSON) |

Tabella B.34: Dettagli chiamata API `/ts/get_test_sequence_feature_test_case_-by_ts_id`.

**Descrizione Parametri:**

- `ts_id` (*int, obbligatorio*): ID della sequenza di test.

## B.35 Recupero Associazioni Test Sequence per ID Feature Case

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_test_sequence_feature_test_case_by_ft_id` | Elenco delle associazioni per il FT indicato (JSON) |

Tabella B.35: Dettagli chiamata API `/ts/get_test_sequence_feature_test_case_by_ft_id`.

**Descrizione Parametri:**

- `ft_id` (*int, obbligatorio*): ID del Feature Test Case.

## B.36 Elenco Tutte le Associazioni Feature Case e Requisito

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_all_feature_test_case_requirement` | Mappatura completa FT-Requisito (JSON) |

Tabella B.36: Dettagli chiamata API `/ts/get_all_feature_test_case_requirement`.

**Descrizione Parametri:**

- *Parametri:* Nessun parametro richiesto.

## B.37 Recupero Associazione Feature Case e Requisito

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_feature_test_case_requirement` | Dettagli dell'associazione specifica (JSON) |

Tabella B.37: Dettagli chiamata API `/ts/get_feature_test_case_requirement`.

**Descrizione Parametri:**

- `ft_id` (*int, obbligatorio*): ID del Feature Test Case.
- `r_id` (*int, obbligatorio*): ID del requisito.

## B.38 Recupero Associazioni Requisito per ID Feature Case

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_feature_-test_case_-requirement_by_ft_id` | Elenco dei requisiti collegati al FT (JSON) |

Tabella B.38: Dettagli chiamata API `/ts/get_feature_test_case_requirement_by_-ft_id`.

**Descrizione Parametri:**

- `ft_id` (*int, obbligatorio*): ID del Feature Test Case.

## B.39 Recupero Associazioni Feature Case per ID Requisito

| Metodo | EndPoint | Risposta |
|---|---|---|
| GET | `/ts/get_feature_-test_case_-requirement_by_r_id` | Elenco dei Feature Case collegati al requisito (JSON) |

Tabella B.39: Dettagli chiamata API `/ts/get_feature_test_case_requirement_by_-r_id`.

**Descrizione Parametri:**

- `r_id` (*int, obbligatorio*): ID del requisito.

## B.40 Aggiornamento Test Sequence

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ts/update_ts` | Esito dell'operazione (JSON) |

Tabella B.40: Dettagli chiamata API `/ts/update_ts`.

**Descrizione Parametri:**

- `id` (*int, obbligatorio*): ID della sequenza di test da aggiornare.
- `changes` (*string, obbligatorio*): Modifiche da applicare (es. stringa JSON o testo con i cambi).

## B.41 Aggiornamento Versione

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ts/update_version` | Esito dell'operazione (JSON) |

Tabella B.41: Dettagli chiamata API `/ts/update_version`.

**Descrizione Parametri:**

- `hash` (*string, obbligatorio*): Hash della versione da modificare.
- `changes` (*string, obbligatorio*): Dati relativi all'aggiornamento.

## B.42 Aggiornamento Associazione Test Sequence e Feature Test

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ts/update_testSequenceFeatureTest` | Esito dell'operazione (JSON) |

Tabella B.42: Dettagli chiamata API `/ts/update_testSequenceFeatureTest`.

**Descrizione Parametri:**

- `ts_id` (*int, obbligatorio*): ID della sequenza di test.
- `ft_id` (*int, obbligatorio*): ID del Feature Test Case.
- `changes` (*string, obbligatorio*): Modifiche apportate alla relazione.

## B.43 Aggiornamento Associazione Feature Test e Requisito

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ts/update_featureTestCaseRequirement` | Esito dell'operazione (JSON) |

Tabella B.43: Dettagli chiamata API `/ts/update_featureTestCaseRequirement`.

**Descrizione Parametri:**

- `ft_id` (*int, obbligatorio*): ID del Feature Test Case.
- `rq_id` (*int, obbligatorio*): ID del requisito.
- `changes` (*string, obbligatorio*): Modifiche apportate alla relazione.

## B.44 Caricamento Dati Coverage

| Metodo | EndPoint | Risposta |
|---|---|---|
| POST | `/ts/add_coverage` | Esito dell'operazione (JSON) |

Tabella B.44: Dettagli chiamata API `/ts/add_coverage`.

**Descrizione Parametri:**

- `override_ts` (*bool, opzionale*): Sovrascrivi i dati relativi alle Test Sequence (Default: True).
- `override_ft` (*bool, opzionale*): Sovrascrivi i dati relativi ai Feature Test Case (Default: True).
- `override_tsft` (*bool, opzionale*): Sovrascrivi le associazioni TS-FT (Default: True).
- `file` (*UploadFile, obbligatorio*): File contenente i dati di coverage da caricare.

# Bibliografia


[1] Sebastián Ramírez. *FastAPI Framework*. URL: https://fastapi.tiangolo.com/.

[2] JFrog. *JFrog Artifactory Documentation*. 2026. URL: https://www.jfrog.com/confluence/.

[3] Sonatype. *Sonatype Nexus Repository Documentation*. 2026. URL: https://help.sonatype.com/en/nexus-repository.html.

[4] Pallets. *Flask Documentation*. 2026. URL: https://flask.palletsprojects.com/.

[5] Python Software Foundation. *Python 3.10.0 Documentation*. 2021. URL: https://docs.python.org/3/whatsnew/3.10.html.

[6] ISO/IEC. *ACID Properties (Atomicity, Consistency, Isolation, Durability)*. URL: https://en.wikipedia.org/wiki/ACID.

[7] European Union Agency for Railways (ERA). *ERTMS Scope of the Test Specifications*. Ver. 3.3.0. Version 3.3.0. 2017. URL: https://www.era.europa.eu/system/files/2023-01/sos3_index037d_-_subset-076-7_v330_era-opi-2017-5.zip.

[8] The Bootstrap Authors. *Bootstrap 5 Documentation*. 2026. URL: https://getbootstrap.com/docs/5.0/getting-started/introduction/.

[9] Encode OSS. *Uvicorn: An ASGI server implementation*. 2026. URL: https://www.uvicorn.org/.

[10] Stuart Knightley et al. *JSZip Documentation*. 2026. URL: https://stuk.github.io/jszip/.

[11] OpenJS Foundation. *jQuery Documentation*. 2026. URL: https://api.jquery.com/.